Breaking ADP-Based Witness Encryption

Muhammad El Gebali, Yaroslav Rebenko, Markus Schofnegger, and Lev Soukhanov

[[alloc] init]
{gebali, yar, markus, lev}@allocinit.xyz

Witness encryption (WE) allows one party to encrypt a message under an arbitrary satisfiable circuit, so that anyone holding a satisfying input can decrypt. Efficient WE enables numerous modern applications, such as identity-based and attribute-based encryption.

Recent candidates for efficient WE base their security on rank properties of structured ciphertext matrices, which encode the validity of a given witness. This shrinks ciphertext sizes considerably compared to previous constructions, but rests on heuristic arguments rather than security reductions.

We describe two attacks against two such constructions, namely the affine determinant program (ADP) construction from 2020 and its arithmetic extension, the AADP, from 2026. The first attack observes that for sparse circuits, the natural regime for both schemes, commutators formed from the public ciphertext matrices have unexpectedly low rank. Elementary linear algebra on these matrices then recovers the encrypted message directly from the public ciphertext, without knowledge of any witness, and hence breaks the security of both schemes. The second attack linearizes the nearly-skew-symmetric (NSS) variant of the ADP construction, recovering the encryption randomness and the message. To our knowledge, ours are the first attacks against these WE candidates, and we verify both in practice.

Contents

1 Introduction 2
1.1 Our Contributions 3
1.2 Related Work 4
2 Preliminaries 5
2.1 Notation and Linear Algebra 5
2.2 Witness Encryption 6
2.3 Affine Determinant Programs 7
2.4 Arithmetic Affine Determinant Programs 9
3 Reducing ADP to AADP 10
4 Linearization Attack on NSS-Based ADP 12
4.1 NSS-Based All-Accept ADP 12
4.2 Linearization Attack on NSS-Based ADP-Based WE 14
5 Commutator Attacks 17

5.1 Discovering Secrets 21
6 Practical Evaluation 23
6.1 The Original ADP Construction 23
6.2 The AADP Construction 25
6.3 The NSS-Based Construction 25
6.4 Practical Results 27
7 Conclusion 28
A General Linearization Attack on WE from CS 30

1 Introduction

Witness encryption (WE) [GGS+13] is an encryption primitive in which a ciphertext is associated with an instance x of an NP language $\mathrm{L} = \{x \in \{0,1\}^* : \exists w \text{ such that } \mathsf{R}(x,w) = 1\}$, where R is an efficiently computable witness relation. Anyone holding a witness w satisfying $\mathsf{R}(x,w) = 1$ can decrypt the ciphertext. Security requires that, when $x \notin \mathrm{L}$, encryptions of any two messages be computationally indistinguishable. Thus, unlike traditional public-key encryption, decryption authority is not associated with possession of a fixed secret key, but with the ability to produce a witness for a computational statement.

WE is both a fundamental cryptographic primitive and a useful building block, enabling applications such as identity-based and attribute-based encryption as well as secret sharing for NP statements [GGS+13]. Despite this broad applicability, known constructions of WE for general NP remain far from practical. Generic constructions based on indistinguishability obfuscation (iO) [JLS21; JLS22; RVV24] or multilinear maps [GGS+13; GLW14] incur large overheads and rely on powerful assumptions. Other direct constructions rely on evasive LWE [VWW22; Tsa22], GapMDP [BIO+20], or ADPs [BIJ+20; SRG+26], whose concrete security is not yet well understood. Consequently, there remains a substantial gap between the theoretical existence of witness encryption and its practical realization.

Among these approaches, the most concrete WE constructions are based on the rank of public matrix programs [BIJ+20; SRG+26], which are called affine determinant programs (ADPs). These techniques determine whether an input $\mathbf{x} = (x_0, \ldots, x_n)$ is in L by evaluating the matrix function $\mathbf{M}(\mathbf{x}) = \sum_{i=0}^{n} x_i \mathbf{M}_i$, and define acceptance through the rank or singularity of $\mathbf{M}(\mathbf{x})$. The ADP framework of [BIJ+20] uses this representation to construct a WE scheme for the NP-complete vector subset sum (VSS) problem. More recently, arithmetic affine determinant programs (AADPs) [SRG+26] extended this approach to arithmetic constraint systems, allowing arithmetic verification procedures, including SNARK verification, to be represented without first translating the entire computation into Boolean constraints.

ADP and AADP are appealing for several reasons. Their ciphertexts consist of explicit matrices over finite fields, their decryption algorithms reduce largely to standard linear algebra, and their concrete costs can be estimated more directly than those of general-purpose obfuscation-based constructions. In particular, the

AADP approach is designed to exploit the native arithmetic structure of SNARK verification and thereby avoid the prohibitive Boolean arithmetization overhead of earlier ADP instantiations. The ADP and AADP frameworks therefore constitute some of the most concrete attempts to obtain general-purpose WE with implementable, or at least concretely estimable, parameters.

This advantage comes with an important limitation. Their security does not rely on a standard cryptographic assumption, and the existing analysis primarily establishes the distribution of a single evaluated matrix and rules out particular classes of attacks. In the WE setting, however, the adversary receives the entire collection of coefficient matrices $\mathbf{M}_0, \ldots, \mathbf{M}_n$. The adversary may evaluate arbitrary field-valued linear combinations of these matrices and compare many correlated evaluations. Consequently, guarantees about the rank distribution of a single evaluation do not by themselves rule out attacks exploiting global algebraic relations among the public coefficients. This makes (A)ADP-based WE schemes natural targets for cryptanalysis.

1.1 Our Contributions

Our goal is to advance the cryptanalytic understanding of (A)ADP-based witness encryption schemes in several directions, with the following results being the main contributions of this paper.

1. We show that the ADP-based WE scheme of [BIJ+20] is a special case of the arithmetic AADP framework of [SRG+26]. In Section 3, we reparameterize the program so that its linear constraints hold by construction, which makes the matrices encoding them cancel and leaves only the component checking the quadratic constraints. This places both schemes in a common framework, which allows us to transfer structural observations and cryptanalytic techniques between them and to analyze both constructions at once.
2. We give an efficient decryption attack in Section 4.2 against the nearly-skew-symmetric (NSS) matrix variant of the ADP-based WE scheme [BIJ+20, Sections 5.1, 6.1.3]. The attack first uses projective inputs at infinity (dropping the first coefficient matrix $\mathbf{M}_0$) to recover most of the hidden masking transformation. It then uses affine inputs to test whether the remaining row can be reconstructed consistently, yielding a distinguisher for the encrypted bit.
3. We give a commutator-based attack in Section 5 that breaks both the ADP- and AADP-based witness encryption schemes when the underlying constraint system is sparse.[1] Here, the constraint system is the circuit encoded by the determinant program, where each variable corresponds to an input and each constraint represents a local gate relation. We call the circuit sparse when each variable appears in only a small number of constraints. This is the natural regime for both constructions, since Boolean bit-checking constraints

[1] An informal description of the commutator approach first appeared in our blog post, `https://www.allocinit.xyz/posts/commutator-attack`.

in ADPs are highly local, while arithmetic circuits used in AADP are typically built from constant fan-in/fan-out and constant low-degree gates, so each wire participates in only a few local relations. We show that this sparsity leaves low-rank relations among the public coefficient matrices, leading to an efficient distinguisher and a full break of the proposed schemes. In particular, the super-constant gate-repetition mitigation suggested in [BIJ+20] does not prevent the attack, since repeating gates leaves the relative sparsity of the constraint system unchanged.
4. We implement the attacks and evaluate their running time on concrete parameter sets. Our experiments show that for the tested instances the resulting distinguishers and attacks are efficient in practice.

Taken together, these attacks cover all ADP-based WE variants described in [SRG+26; BIJ+20], and to the best of our knowledge they give the first break of the WE scheme of [BIJ+20], whose iO counterpart was broken in [YCY22]. Our techniques identify concrete structural weaknesses in WE schemes based on ADPs and AADPs. Future similar constructions must ensure not only that each individual evaluation has the intended rank distribution, but also that the public coefficient matrices do not reveal exploitable algebraic relations when analyzed jointly.

1.2 Related Work

ADPs were introduced in [BIJ+20] as a framework for both iO and WE. Separate candidate constructions were proposed for these two primitives, together with an analysis of several potential algebraic attacks against them. In particular, that analysis covers attacks based on determinant interpolation, correlated spans, linearization, and evaluations on non-Boolean inputs. These considerations provided heuristic support for the proposed constructions, but did not establish security under a standard cryptographic assumption.

A cryptanalytic attack against the ADP-based iO candidate was later given in [YCY22]. That attack exploits a weakness in one of the randomization steps of the candidate and applies to a fairly general class of programs. In particular, the random local substitutions introduced in [BIJ+20] to thwart earlier attack strategies do not prevent it. The attack is aimed at the iO candidate, however, and does not directly apply to the WE construction of [BIJ+20], which is a separate candidate with a different program structure and evaluation procedure. Thus, prior work broke the proposed ADP-based iO candidate, but did not give a corresponding break of the ADP-based WE scheme.

With the introduction of AADP [SRG+26], the cryptanalysis of rank-based WEs was revisited in the arithmetic setting. That analysis considers correlated span attacks when multiple projective solutions are available, parametric rank drop searches, and algebraic attacks such as linearization, extending the linearization attacks of [BIJ+20]. It is argued that projective safety prevents both correlated span and parametric rank drop attacks, and experimental evidence is reported suggesting that the remaining attacks are infeasible for the proposed

parameters. Nevertheless, the resulting security claim remains heuristic and is not based on a reduction from a standard assumption.

One of the attacks we present makes heavy use of commutators, so we briefly note where this tool has appeared before. The algebraic relations we exploit go back to Strassen's equations for tensor rank [Str83], which were obtained by taking commutators of matrices formed from the slices of a tensor, and which remain a standard source of lower bounds in algebraic complexity theory. In cryptography, commutators appear on the design side in the Anshel–Anshel–Goldfeld commutator key exchange, whose security rests on the commutator key exchange problem [Tsa15]. By contrast, our attack places commutators on the attacker's side, as a distinguisher built from quotients of the public matrices.

2 Preliminaries

We first give an overview of the notation used in the paper and recall a few linear algebra properties that our attacks rely on. We then recall the definition of witness encryption and the two constructions that we consider in our analysis, namely affine determinant programs (ADP) of [BIJ+20] and their arithmetic variant (AADP) of [SRG+26].

2.1 Notation and Linear Algebra

We work over a finite field $\mathbb{F}_p$ for a prime p. We use λ for the security parameter and $\mathrm{negl}(\lambda)$ for a negligible function. Vectors are denoted by lowercase bold letters $\mathbf{x}$ and matrices by uppercase bold letters $\mathbf{M}$, where $\mathbf{M}^\top$ denotes the transpose of $\mathbf{M}$. We write $\mathbf{e}_i$ for the i-th standard basis vector, $\mathbf{I}_k$ for the $k \times k$ identity matrix, and $\langle \mathbf{u}, \mathbf{v} \rangle$ for the standard inner product. Concatenation of vectors is denoted by $(\mathbf{u} \mid \mathbf{v})$, $\circ$ denotes the Hadamard (entry-wise) product of matrices, and $\otimes$ denotes the Kronecker (tensor) product of matrices. Notation on matrices extends to vectors viewed as matrices with a single column. For a positive integer t we write $[t] \coloneqq \{1, \ldots, t\}$. Tensor slices, matrix rows and columns, vector coordinates, gates, and samples are indexed starting from 0. For witness variables, a homogenized input is $\mathbf{x} = (x_0, x_1, \ldots, x_n)$ whose 0-th coordinate is the homogenizing variable, so variable (wire) indices run over $[n]$.

Our results make heavy use of ranks and kernels of structured matrices over $\mathbb{F}_p$. For a matrix $\mathbf{X} \in \mathbb{F}_p^{a \times b}$ we write $\mathrm{rank}(\mathbf{X})$ for its rank, $\ker(\mathbf{X}) \coloneqq \{\mathbf{v} : \mathbf{X}\mathbf{v} = \mathbf{0}\}$ for its (right) kernel, and we refer to $\ker(\mathbf{X}^\top)$ as its left kernel. For two matrices $\mathbf{X}, \mathbf{Y}$ over $\mathbb{F}_p$ of compatible dimensions, we have that

(1) $\mathrm{rank}(\mathbf{XY}) \leq \min\{\mathrm{rank}(\mathbf{X}), \mathrm{rank}(\mathbf{Y})\}$,
(2) $\mathrm{rank}(\mathbf{X} + \mathbf{Y}) \leq \mathrm{rank}(\mathbf{X}) + \mathrm{rank}(\mathbf{Y})$, and
(3) $\mathrm{rank}(\mathbf{X} \otimes \mathbf{Y}) = \mathrm{rank}(\mathbf{X}) \cdot \mathrm{rank}(\mathbf{Y})$.

We use $[\mathbf{X}, \mathbf{Y}] \coloneqq \mathbf{XY} - \mathbf{YX}$ to denote the *commutator* of two square matrices. The commutator is bilinear and vanishes if and only if the two matrices commute. Additionally, properties *(1)* and *(2)* above imply that

$$\mathrm{rank}([\mathbf{X}, \mathbf{Y}]) \leq 2 \min\{\mathrm{rank}(\mathbf{X}), \mathrm{rank}(\mathbf{Y})\}.$$

We write $\mathbf{M}_{\ell,i,j}$ for the (i,j)-th entry of the ℓ-th matrix slice $\mathbf{M}_\ell$ of an order-3 tensor $\mathbf{M}$. In other words, the indices are a tuple (*slice*, *row*, *column*) in that order. We write $\mathbf{M}_{\cdot,\cdot,j}$ for the (horizontal) concatenation of the j-th column of all slices, and generally use $\cdot$ to denote a free index. We omit $\cdot$ at trailing indices, e.g., $\mathbf{M}_{\cdot,i}$ is the (vertical) concatenation of the i-th row of all slices and $\mathbf{M}_\ell$ is the ℓ-th slice. We write $i_1 : i_2$ for indices in the inclusive range $\{i_1, i_1+1, \ldots, i_2\}$. The same applies to a matrix with indices as a pair (*row*, *column*). For a vector $\mathbf{x}$, we denote by x_i the i-th entry.

Throughout, n denotes the number of witness variables (the input length of the program) and m the number of gates, i.e., the number of constraints in the underlying constraint system.

We use $\mathbf{M}(\mathbf{x})$ for the matrix-valued map evaluated on an input $\mathbf{x}$. We do not distinguish between a vector $\mathbf{u} \in \mathbb{F}_p^{n+1}$ and a linear function on $\mathbb{F}_p^{n+1}$, since finite-dimensional vector spaces are isomorphic to their dual spaces. We therefore occasionally write $\mathbf{u}(\mathbf{x})$ to mean $\langle \mathbf{u}, \mathbf{x} \rangle$.

We let $\mathrm{supp}(i) \subseteq \{0, \ldots, m-1\}$ denote the *support* of the i-th variable, i.e., the set of gate indices that depend on variable i. We denote by $\deg(i) \coloneqq |\mathrm{supp}(i)|$ the *degree* of variable i, i.e., the number of gates that depend on variable i.

The constructions we study inject randomness in order to push the rank of a ciphertext matrix up to its maximal (generic) value. We will use the standard fact that a matrix whose entries are chosen uniformly at random in $\mathbb{F}_p$ attains its generic rank except with probability that vanishes as p grows. More generally, a matrix whose entries are evaluations of fixed nonzero polynomials at uniformly random points is of full (generic) rank except with negligible probability $\mathrm{negl}(\lambda)$ by the Schwartz–Zippel lemma.

2.2 Witness Encryption

Witness encryption [GGS+13] allows one to encrypt a message under a statement x of an NP language L, so that decryption is possible exactly for parties holding a witness w for $x \in \mathrm{L}$. We recall the definition here and refer to [BIJ+20; GGS+13] for a more detailed overview.

Definition 1 (Witness Encryption). *A witness encryption scheme for an* NP *language* L *with witness relation* R *consists of two polynomial-time algorithms:*

- $\mathsf{Enc}(1^\lambda, x, b)$ *takes a security parameter* 1^λ*, a statement* x*, and a message* b*, and outputs a ciphertext* ct*.*
- $\mathsf{Dec}(\mathsf{ct}, w)$ *takes a ciphertext* ct *and a candidate witness* w*, and outputs a message or the symbol* $\perp$*.*

The scheme is correct *if for every* $x \in \mathrm{L}$ *and every* w *with* $(x, w) \in \mathsf{R}$*,*

$$\Pr\big[\mathsf{Dec}(\mathsf{Enc}(1^\lambda, x, b), w) = b\big] \geq 1 - \mathrm{negl}(\lambda).$$

It satisfies soundness security *if for every* $x \notin \mathrm{L}$ *and every* PPT *adversary* $\mathcal{A}$*,*

$$\big|\Pr\big[\mathcal{A}(\mathsf{Enc}(1^\lambda, x, 0)) = 1\big] - \Pr\big[\mathcal{A}(\mathsf{Enc}(1^\lambda, x, 1)) = 1\big]\big| \leq \mathrm{negl}(\lambda).$$

2.3 Affine Determinant Programs

An affine determinant program encodes a Boolean function as the determinant of an affine matrix-valued map [BIJ+20].

Definition 2 (Affine Determinant Program [BIJ+20]). *An* affine determinant program $\mathsf{ADP} : \{0,1\}^n \to \{0,1\}$ *of width* k *over* $\mathbb{F}_p$ *is specified by a tuple of matrices* $(\mathbf{A}, \mathbf{B}_1, \ldots, \mathbf{B}_n) \in \left(\mathbb{F}_p^{k\times k}\right)^{n+1}$ *and an evaluation function* $\mathsf{Eval} : \mathbb{F}_p \to \{0,1\}$*. On input* $\mathbf{x} \in \{0,1\}^n$ *it outputs*

$$\mathsf{Eval}\left(\det\left(\mathbf{A} + \sum_{i\in[n]} x_i \mathbf{B}_i\right)\right), \quad \textit{where} \quad \mathbf{M}(\mathbf{x}) \coloneqq \mathbf{A} + \sum_{i\in[n]} x_i \mathbf{B}_i.$$

Natural choices are $\mathsf{Eval}_0(y) = 1$ *if* $y = 0$ *(and* 0 *otherwise) and its complement* $\mathsf{Eval}_{\neq 0} = 1 - \mathsf{Eval}_0$*, so that the program decides whether* $\mathbf{M}(\mathbf{x})$ *is singular or, respectively, of full rank. The witness encryption candidate below relies exactly on this rank test.*

Witness Encryption from ADPs. The construction of [BIJ+20, Section 5.2] realizes witness encryption for the NP-complete (vector) subset sum language and extends to all of NP under standard NP reductions that also map witnesses, as instantiated in [BIJ+20, Section 6.1.1]. The ciphertext is an ADP $\mathbf{M} = \mathbf{M}_{\mathrm{AA}} + \mathbf{M}_{\mathrm{VSS}} + (b\mathbf{S}, \mathbf{0}, \ldots, \mathbf{0})$ that superposes three parts, namely an encoding $\mathbf{M}_{\mathrm{VSS}}$ of the subset sum statement, a randomized *all-accept* program $\mathbf{M}_{\mathrm{AA}}$ that acts as structured noise by being full rank on every non-binary input, and the message bit b added through a uniformly random matrix $\mathbf{S} \in \mathbb{F}_p^{k\times k}$ in the constant term $\mathbf{A}$. For a valid binary witness $\mathbf{w}$, the matrix $\mathbf{M}(\mathbf{w})$ is rank-deficient when $b = 0$ and full rank when $b = 1$, so decryption recovers the bit as $\mathsf{Eval}_{\neq 0}(\det(\mathbf{M}(\mathbf{w}))) = b$. For a false statement, no witness exists and the all-accept noise is conjectured to keep b hidden, which is the heuristic basis of the scheme's soundness. We refer to [BIJ+20] for the precise instantiation, but we motivate the construction here for clarity.[2]

We will describe how the constraints are encrypted in the following order.

- We start with $\varepsilon = 0$ and $q = n^{\varepsilon} = 1$, meaning that no super-constant rank mitigation is applied, in order to understand the base construction.
- We describe how to encrypt the linear constraints described in the VSS instance.
- We describe how the quadratic bit-check constraints are encoded to enforce that the solutions are binary. We do it only for the formula-based variant

[2] Notice that there are two typos in the referenced construction.

1. For $\ell \in [n]$, we should have $\mathbf{B}_\ell = \ldots + \mathbf{S}_\ell \mathbf{T}_\ell^\top + \ldots$, so $\mathbf{V}_\ell$ was repeated by mistake (see the correctness argument again for verification).
2. There are only $n + 1$, not $2n[+1]$, matrices $\mathbf{R}_\ell$, and for $\ell \in n + [n]$, we have $\mathbf{B}_\ell = \ldots + \mathbf{R}_{\ell - n}$ instead of $\mathbf{R}_\ell$, as otherwise the scheme is not even correct.

[BIJ+20, Section 5.2] and defer the NSS variant [BIJ+20, Section 5.1] to Section 4.1.
- We describe the effect of choosing the super-constant parameter $\varepsilon > 0$ on the program.

We will see that the size of ADP matrices is driven by the rank of the (quadratic) AA part alone. Hence, in what follows, we denote by m the number of *quadratic* constraints. Let $k \coloneqq m+1$ be the width of the ADP, which ensures that only valid solutions achieve rank $m = k-1$ on m satisfied gates, rendering the ADP rank-deficient. If a single gate is broken, the rank is at least k and the ADP is full-rank. We will update the concrete value of m as we build up the ADP construction.

First, consider the VSS part, which encrypts the linear system of constraints $\mathbf{Hw} = \boldsymbol{\ell}$. The matrix $\mathbf{H} \in \mathbb{F}_p^{(n+1)\times 2n}$ and the vector $\boldsymbol{\ell} \in \mathbb{F}_p^{n+1}$ have the form

$$\mathbf{H} \coloneqq \begin{pmatrix} \mathbf{h}^\top & \mathbf{0}^\top \\ \mathbf{I}_n & \mathbf{I}_n \end{pmatrix} \quad \text{and} \quad \boldsymbol{\ell} \coloneqq \begin{pmatrix} \ell \\ 1 \\ 1 \\ \vdots \\ 1 \end{pmatrix},$$

so that the 0-th row encodes the subset sum equation $\langle \mathbf{h}, \mathbf{w} \rangle = \ell$ for $\mathbf{w} \in \mathbb{F}_p^n$, and the i-th row checks that $w_{i+n} = 1 - w_i$ for $i \in [n]$. In other words, if $w_i \in \{0,1\}$ is a bit, then w_{i+n} is intended to be its binary negation (complement).

To encode the j-th linear constraint (equation) on $\mathbf{w}$, sample a random matrix $\mathbf{R}_j \leftarrow \mathbb{F}_p^{k\times k}$ and add $-\ell_j \mathbf{R}_j$ to the matrix $\mathbf{A}$, which acts as the coefficient of the homogenizing variable w_0, and $\mathbf{H}_{j,i}\mathbf{R}_j$ to $\mathbf{B}_i$, which carries the coefficient of w_i. Since there are $n+1$ rows (equations), we need exactly $n+1$ such matrices.

Notice that, up to this point, $m = 0$, and the VSS program on its own can use random scalars to mask the encrypted bit.

Remark 1. The VSS ADP on its own is secure for an inconsistent linear system. Consider an instance of subset sum where $\mathbf{h} = \mathbf{0}$ and $\ell = 1$, so that there are no affine solutions even in the algebraic closure. Then, the VSS ADP is effectively a single matrix $\mathbf{A} = \mathbf{R}_0 + b\mathbf{S}$ (see Section 3), which is a one-time pad (OTP) encryption of the message $b \in \{0,1\}$ and is secure for a single use. The results in this paper are meant to show that the WE schemes in [SRG+26; BIJ+20] are not secure as described for more meaningful instances, namely those where the linear constraints are consistent, while the ideal of solutions to the full NP instance (including the quadratic constraints) may still be trivial or finding any non-trivial solution may be NP-hard.

To get a WE for all of NP, the description of the problem has to be completed to the NP-complete subset sum problem, which requires the supplied witnesses to be binary. We refer the reader to Section 3 for the full description and only motivate it here. A bit-check gate $\mathbf{G}_i \in \mathbb{F}_p^{2\times 2}$ ensures that $\operatorname{rank}(\mathbf{G}_i)$ is 1 if $w_i \in \{0,1\}$ and 2 otherwise. Then those gates are randomized using $\mathbf{P}_i, \mathbf{Q}_i^\top \leftarrow \mathbb{F}_p^{k\times 2}$

in the same way gates are embedded in the AADP construction [SRG+26], as we will explain in the next section.[3] Those matrices are built from the matrices $\mathbf{U}_i, \mathbf{V}_i, \mathbf{S}_i, \mathbf{T}_i$ in the original construction. This part requires $m = 2n$, since there is one bit-check gate for each w_i with $i \in [2n]$, i.e., both the input variables and their complements are simultaneously bit-checked. Had only the wires for $i \in [n]$ been checked, relying on the VSS consistency check to ensure their complements are also Boolean, the program would have had half the size, i.e., $m = n$.[4]

Finally, it was suggested in [BIJ+20, Section 7.1.2] to make the rank budget super-constant in the number of gates/wires in order to mitigate the risk of a successful linearization attack. This is achieved by repeating *every* (bit-check) gate q times for $q = n^{\varepsilon} = \omega_n(1)$, where $\varepsilon := \varepsilon(n) \in (0, 1)$ is chosen by the implementation to ensure that the cost of a linearization attack exceeds an adversary $\mathcal{A}$'s prescribed computational bounds, e.g., λ bits of work. This immediately translates to a number of gates $m = 2nq = 2n^{1+\varepsilon}$, and it concludes the construction and the choice of instance size in [BIJ+20, Section 5.2].

As we will see in Section 5, this mitigation is insufficient (and even inconsequential) against the commutator attack, and it is possible to mount the attack regardless of the choice of ε. In other words, repeating the gates in this naive fashion does not change the relative sparsity of the bit-check gates with respect to the whole instance, still leaving enough room for a distinguisher to succeed independently of ε.

2.4 Arithmetic Affine Determinant Programs

The arithmetic variant of [SRG+26] replaces the bit-valued inputs of Definition 2 by field-element inputs and the bit-checking constraints by a more general system of quadratic constraints.

Definition 3 (Arithmetic ADP [SRG+26]). *An* arithmetic affine determinant program $\mathsf{AADP} : \mathbb{F}_p^n \to \{0, 1\}$ *is specified by square matrices* $\mathbf{M}_0, \ldots, \mathbf{M}_n$ *over* $\mathbb{F}_p$. *On a nonzero input* $\mathbf{x} \in \mathbb{F}_p^n$ *it outputs*

$$\mathsf{Eval}_0\left(\det\left(\mathbf{M}_0 + \sum_{i\in[n]} x_i \mathbf{M}_i\right)\right),$$

where $\mathsf{Eval}_0(y) = 1$ *if and only if* $y = 0$.

Projectively Safe Constraint Systems. The statements encoded by AADPs are quadratic constraint systems. Given matrices $\mathbf{A}, \mathbf{B}, \mathbf{C}, \mathbf{D} \in \mathbb{F}_p^{m\times(n+1)}$, one considers, for each gate $j \in \{0, \ldots, m-1\}$, the constraint

$$\langle \mathbf{A}_j, \mathbf{x}\rangle \cdot \langle \mathbf{B}_j, \mathbf{x}\rangle = \langle \mathbf{C}_j, \mathbf{x}\rangle \cdot \langle \mathbf{D}_j, \mathbf{x}\rangle,$$

[3] We replaced $\mathbf{L}_i$ and $\mathbf{R}_i$ from [SRG+26] with $\mathbf{P}_i$ and $\mathbf{Q}_i$, respectively, to avoid notational conflicts with matrices in the VSS ADP, and to match the original description in [BIJ+20, Section 5.2].

[4] Including q, however, this would not have had an effect on the ciphertext size, as it would probably be absorbed in ε to achieve the same security margin.

in the homogenizing variable x_0 and the variables $x_1, \ldots, x_n$, where $\mathbf{A}_j$ denotes the j-th row. A nonzero solution with $x_0 = 0$ is called a *solution at infinity*. The construction requires the constraint system to be *projectively safe*, a structural condition on the constraints that guarantees that the only solution with $x_0 = 0$ is the trivial solution $\mathbf{x} = \mathbf{0}$ [SRG+26]. In particular, bit-checked systems, and hence the construction of [BIJ+20], are automatically projectively safe.

Witness Encryption from AADPs. A generation procedure $\mathsf{AADP.Gen}$ maps a projectively safe constraint system $(\mathbf{A}, \mathbf{B}, \mathbf{C}, \mathbf{D})$ with n variables and m gates to an AADP $\mathbf{M}$ of dimension $k \times k$, realized as a linear-combination-valued matrix $\mathbf{M}(\mathbf{x}) = \sum_{i=0}^{n} x_i \mathbf{M}_i$. Encryption of a message $\mathtt{msg} \in \mathbb{F}_p$ adds $\mathtt{msg}$ to the bottom-right entry of the constant coefficient matrix $\mathbf{M}_0$. Decryption evaluates the ciphertext at a solution $\mathbf{x}$ of the constraint system and recovers $\mathtt{msg}$ as the unique value of t for which subtracting t from that same entry makes the matrix singular [SRG+26, Section 2.4]. The determinant is indeed linear in t and the equation is efficiently solvable once the corresponding cofactor in the adjugate matrix is computed.

A gate matrix $\mathbf{U}_j \in \left(\mathbb{F}_p^{n+1}\right)^{4\times 4}$ is defined as

$$\mathbf{U}_j(\mathbf{x}) \coloneqq \begin{pmatrix} \mathbf{A}_j(\mathbf{x}) & \mathbf{C}_j(\mathbf{x}) & -\boldsymbol{\Xi}_j(\mathbf{x}) & 0 \\ \mathbf{D}_j(\mathbf{x}) & \mathbf{B}_j(\mathbf{x}) & 0 & \boldsymbol{\Xi}_j(\mathbf{x}) \\ 0 & 0 & \mathbf{B}_j(\mathbf{x}) & \mathbf{C}_j(\mathbf{x}) \\ 0 & 0 & \mathbf{D}_j(\mathbf{x}) & \mathbf{A}_j(\mathbf{x}) \end{pmatrix}.$$

The coefficients $\boldsymbol{\Xi} \leftarrow \mathbb{F}_p^{m\times(n+1)}$ are chosen at random and are intended to guarantee the rank is at least 2 even when the gate is degenerate, i.e., $\mathbf{A}_j = \mathbf{B}_j = \mathbf{C}_j = \mathbf{D}_j = \mathbf{0}$. The full public tensor is then produced from gate randomizations as

$$\mathbf{M}(\mathbf{x}) = \sum_{j=0}^{m-1} \mathbf{L}_j \mathbf{U}_j(\mathbf{x}) \mathbf{R}_j + \mathtt{msg}\, \mathbf{e}_{k-1} \mathbf{e}_{k-1}^{\top},$$

where $\mathbf{L}_j, \mathbf{R}_j^{\top} \leftarrow \mathbb{F}_p^{k\times 4}$ are the randomizations of the j-th gate.

It is argued in [SRG+26] that the rank is 2 (w.h.p.) if the gate is satisfied and 4 otherwise. Consequently, the matrix size is chosen as $k = 2m+1$ to ensure it is rank-deficient only on valid witnesses to the projective constraint system and full-rank otherwise. The construction also embeds a pairing-based general-purpose SNARK verifier for NP [SRG+26, Section 4], proving that the resulting WE construction is indeed for all of NP.

3 Reducing ADP to AADP

We recall that the ADP-based WE construction of [BIJ+20, Section 5.2] combines a program enforcing the public Vector Subset Sum (VSS) linear constraints with an all-accept program enforcing the Booleanity of the witness coordinates. We describe the construction briefly in Section 2.3. Our goal is to reparameterize

the VSS-based ADP so that the public VSS linear constraints hold identically, allowing us to isolate the all-accept (AA) component. We present this transformation to show that the VSS-based ADP construction can be rewritten as a special case of the AADP framework, which allows the attack in Section 5 to be analyzed in a unified setting and applied to both schemes. Note that this transformation annihilates the VSS component regardless of how the AA program is built. In particular, it is used again in Section 4.1 to mount the linearization attack on the NSS variant in Section 4.2.

Reduction to the All-Accept Component. Consider the VSS instance defined in [BIJ+20, Section 5.2], i.e., given public inputs $\ell \in \mathbb{F}_p$ and $\mathbf{h} \in \mathbb{F}_p^n$, find a witness $\mathbf{w} \in \mathbb{F}_p^n$ such that $\langle(-\ell \mid \mathbf{h}), (w_0 \mid \mathbf{w})\rangle = 0$. Typically, $w_0 = 1$ for affine solutions, but it is possible to supply $w_0 = 0$ to get solutions at infinity. To absorb the public VSS linear constraint into the program, we explicitly parameterize all witnesses satisfying it. A concrete way to do this is to find a particular vector $\mathbf{w}^\star$ that satisfies the affine equation $\langle \mathbf{h}, \mathbf{w}\rangle = \ell$[5] after setting $w_0 = 1$, and a matrix $\mathbf{V} \in \mathbb{F}_p^{n\times(n-1)}$ such that $\mathbf{h}^\top \mathbf{V} = \mathbf{0}^\top$, i.e., the solutions to the homogeneous system, which are exactly the solutions at infinity. From these we build a linear transformation $\mathbf{P} \in \mathbb{F}_p^{(2n+1)\times n}$ of the form

$$\mathbf{P} \coloneqq \begin{pmatrix} 1 & \mathbf{0}^\top \\ \mathbf{w}^\star & \mathbf{V} \\ \mathbf{1} - \mathbf{w}^\star & -\mathbf{V} \end{pmatrix},$$

and consider the new program defined as $\widetilde{\mathbf{M}}(\mathbf{z}) \coloneqq \mathbf{M}(\mathbf{P}\mathbf{z})$. Since $\mathbf{M}_{\mathrm{VSS}}(\mathbf{P}\mathbf{z}) = \mathbf{0}$, we get a new program parametrized by the linear subspace of solutions to the linear constraints. The new program has the following properties.

1. Any basis of the kernel of the VSS system could work, but we make this particular choice to ensure $z_0 = w_0$, so the message is still encrypted in the same position. In particular, all other coefficient matrices $\widetilde{\mathbf{B}}_i$, the matrix coefficients of z_i in the modified program $\widetilde{\mathbf{M}}$, have no message component.
2. We recall that the matrix $\mathbf{R}_j$ encodes the j-th VSS (linear) constraint. Since every vector in the image of $\mathbf{P}$ satisfies $(-\boldsymbol{\ell} \mid \mathbf{H})\mathbf{P}\mathbf{z} = \mathbf{0}$, the coefficient multiplying each $\mathbf{R}_j$ evaluates to zero after substituting $(w_0, \mathbf{w}, w_0\mathbf{1} - \mathbf{w}) = \mathbf{P}\mathbf{z}$. Hence, the entire VSS component disappears from $\mathbf{M}(\mathbf{P}\mathbf{z})$, and the transformed program contains only the $\mathbf{U}_j, \mathbf{V}_j, \mathbf{S}_j, \mathbf{T}_j$ terms belonging to the AA component.
3. The random $\mathbf{c}$ part ensures that the rank is at least $2n^{1+\varepsilon}$ except with negligible probability.
4. Since w_j is binary if and only if $1 - w_j$ is, we get an extra rank of $2n^\varepsilon$ for every supplied non-binary input and 0 otherwise (see the correctness argument for the AA formula-based variant in [BIJ+20] for a better intuition on how either the row or the column spaces coincide with already existing terms).

[5] This procedure works as long as $\mathbf{h} \neq \mathbf{0}$ (see also Remark 1).

Therefore, we end up with a smaller program that an adversary can construct themselves from public matrices. Its rank is an integer in the interval $2[n^{1+\varepsilon}, 2n^{1+\varepsilon}]$, and it is rank-deficient (except with negligible probability) if and only if all supplied inputs are binary, i.e., $\mathbf{w} \in \{0,1\}^n$, and the encrypted message is $b = 0$. Hence, a valid binary input in the image of $\mathbf{P}$ can distinguish and decrypt the encrypted bit.

Finally, if we reformulate the formula-based AA program in the form

$$\begin{aligned}
&\forall i \in [2n],\\
\mathbf{P}_i &\coloneqq \left(\mathbf{S}_i \middle| \mathbf{U}_i\right) \in \mathbb{F}_p^{(2n^{1+\varepsilon}+1)\times 2n^{\varepsilon}},\\
\mathbf{Q}_i &\coloneqq \left(\frac{\mathbf{T}_i^\top}{\mathbf{V}_i^\top}\right) \in \mathbb{F}_p^{2n^{\varepsilon}\times(2n^{1+\varepsilon}+1)},\\
\mathbf{c}_i &\leftarrow \mathbb{F}_p^{2n},\\
\mathbf{G}_i^{\mathrm{bit}}(\widetilde{\mathbf{w}}) &\coloneqq \begin{pmatrix} \widetilde{w}_i & \langle \mathbf{c}_i, \widetilde{\mathbf{w}}\rangle \\ 0 & 1-\widetilde{w}_i \end{pmatrix} \in \left(\mathbb{F}_p^{2n}\right)^{2\times 2},\\
\mathbf{G}_i(\widetilde{\mathbf{w}}) &\coloneqq \mathbf{G}_i^{\mathrm{bit}}(\widetilde{\mathbf{w}}) \otimes \mathbf{I}_{n^{\varepsilon}} \in \left(\mathbb{F}_p^{2n}\right)^{2n^{\varepsilon}\times 2n^{\varepsilon}},\\
\mathbf{M}((1 \mid \widetilde{\mathbf{w}})) &\coloneqq \sum_{i\in[2n]} \mathbf{P}_i \mathbf{G}_i(\widetilde{\mathbf{w}}) \mathbf{Q}_i \in \left(\mathbb{F}_p^{2n}\right)^{(2n^{1+\varepsilon}+1)\times(2n^{1+\varepsilon}+1)},
\end{aligned}$$

we get exactly the construction in [BIJ+20, Section 5.2] but in the language of the AADP framework of [SRG+26].[6] The gates are just bit checks of all variables and their complements. The $\mathbf{c}$ mask is instead denoted $\boldsymbol{\xi}$ in [SRG+26]. The repetition of gates n^{ε} times is exactly the super-constant rank mitigation discussed in [SRG+26; BIJ+20] against linearization attacks. The encryption is done as in [BIJ+20] by adding $b\mathbf{S}$ to $\mathbf{A}$, which corresponds to the coefficient matrix of the homogenizing variable w_0.

4 Linearization Attack on NSS-Based ADP

In addition to the formula-based all-accept formulation, an alternative approach to build an ADP that accepts only binary inputs with high probability was devised in [BIJ+20, Sections 5.1, 6.1.3].

4.1 NSS-Based All-Accept ADP

NSS Matrices and Their Quadratic Forms. Recall that a *skew-symmetric* (SS) matrix is a square matrix $\mathbf{S}$ with the property that $\mathbf{S}_{i,j} + \mathbf{S}_{j,i} = 0$ for all i, j. In [BIJ+20, Section 5.1], a *nearly-skew-symmetric* (NSS) matrix is defined to be a square matrix $\mathbf{N}$ that instead satisfies $\mathbf{N}_{i,j} + \mathbf{N}_{j,i} = 0$ for $j > i > 0$ and

[6] We replace the gate matrix $\mathbf{U}$ with $\mathbf{G}$ to avoid confusion with the notation of [BIJ+20]. See also Footnote 3 for other changes.

$\mathbf{N}_{i,0} + \mathbf{N}_{i,i} + \mathbf{N}_{0,i} = 0$ for all i.[7] In particular, *only* $\mathbf{N}_{0,0}$ is necessarily 0 for NSS matrices, while SS matrices have all diagonal entries identically 0. Let $k = n+1$ and $\mathbf{N} \in \mathbb{F}_p^{k\times k}$, and consider the quadratic form $\mathfrak{Q}^{\mathbf{N}}(\mathbf{x}) \coloneqq \mathbf{x}^\top \mathbf{N}\mathbf{x}$ for a vector $\mathbf{x} \in \mathbb{F}_p^k$ so it expands to

$$\sum_{i,j=0}^{n} \mathbf{N}_{i,j} x_i x_j = \sum_{i=0}^{n} \left[\underbrace{(\mathbf{N}_{i,0} + \mathbf{N}_{0,i})}_{=-\mathbf{N}_{i,i}} x_i x_0 + \mathbf{N}_{i,i} x_i^2 \right] + \sum_{n \geq j > i > 0} \underbrace{(\mathbf{N}_{i,j} + \mathbf{N}_{j,i})}_{=0} x_i x_j$$

$$= \sum_{i=1}^{n} \mathbf{N}_{i,i}(x_i^2 - x_i x_0), \tag{1}$$

where the second sum cancels by the first property of NSS and we make the replacement $\mathbf{N}_{i,0} + \mathbf{N}_{0,i} = -\mathbf{N}_{i,i}$ using the second property. Notice that the term at $i = 0$ vanishes since $\mathbf{N}_{0,0} = 0$.

Therefore, the quadratic form $\mathfrak{Q}^{\mathbf{N}}(\cdot)$ acts as a single LPCP-style random query for the bit-check constraints as long as the diagonal entries of $\mathbf{N}$ are chosen i.i.d. uniformly and independently of $\mathbf{x}$. Using several i.i.d. uniform NSS matrices $\mathbf{N}_\ell$, the quadratic forms $\mathfrak{Q}^{\mathbf{N}_\ell}(\cdot)$ act as independent LPCP-style queries for bit-check constraints over the range of ℓ.[8]

The formula-based AA ADP is then replaced with an NSS-based one. We will use a tensor description equivalent to the one in [BIJ+20, Section 5.1], as we believe it makes the vulnerability and the attack easier to understand. We refer the interested reader to [BIJ+20, Sections 5.1, 6.1.3] for the original description.

NSS-Based All-Accept ADP. Let $\mathrm{NSS}(p, n)$ be the set of NSS matrices in $\mathbb{F}_p^{k\times k}$ for $k \coloneqq 2n + 1$.[9] Sample an invertible matrix $\mathbf{T} \leftarrow \mathbb{F}_p^{k\times k}$ and k matrices $\mathbf{N}_\ell \leftarrow \mathrm{NSS}(p, n)$. Together, the $\mathbf{N}_\ell$ form an order-3 tensor of dimension $k \times k \times k$ in which $\mathbf{N}_\ell$ is the ℓ-th slice. Define another tensor $\mathbf{C}$ such that the ℓ-th slice of $\mathbf{C}$ is $\mathbf{C}_\ell \coloneqq \mathbf{T}^{-1}\mathbf{N}_\ell$, i.e., each matrix slice is left randomized by the *same* random linear transformation $\mathbf{T}^{-1}$. Define k coefficient matrices $\mathbf{M}_0, \ldots, \mathbf{M}_{k-1} \in \mathbb{F}_p^{k\times k}$ and regard them as an order-3 tensor $\mathbf{M} \in \mathbb{F}_p^{k\times k\times k}$ where $\mathbf{M}_j$ denotes the j-th slice of $\mathbf{M}$. We define $\mathbf{M}_{j,\cdot,\ell} \coloneqq \mathbf{C}_{\ell,\cdot,j}$, i.e., the j-th column of the ℓ-th matrix slice of $\mathbf{C}$ becomes the ℓ-th column of the j-th slice of $\mathbf{M}$. Said differently, $\mathbf{M}$ is obtained from $\mathbf{C}$ by a tensor transposition that swaps the slice and column indices and keeps the row index fixed. Notice that in [BIJ+20], $\mathbf{A} = \mathbf{M}_0$ and $\mathbf{B}_j = \mathbf{M}_j$.

[7] In [BIJ+20], it was described with 1-based indexing, but we just reformulate it to 0-based indexing here for consistency of notation with the rest of the paper.

[8] It is possible to generalize this idea to any quadratic constraints and it is not hard to see that the techniques in Section 4.2 extend without any substantial changes.

[9] The original description [BIJ+20, Section 5.1] is for $k = n + 1$, but we make this change to make it compatible with the VSS ADP as described in Section 3 at $q = 1$. This was suggested as a potential mitigation to linearization attacks for random recovery in [BIJ+20, Section 7.2]. The definition works as is for any choice of $k \coloneqq k(n)$ in fact.

The NSS-based AA program is used with two message-embedding mechanisms in [BIJ+20, Section 5.1]. The first is the same mechanism used for the formula-based AA construction, where a message bit is embedded by adding an independent random matrix to the constant coefficient matrix. The second is the witness key-encapsulation mechanism [CV21] (WKEM) described in [BIJ+20, Section 6.1.3]. The latter is almost identical to the encryption and decryption procedures of [SRG+26], and both are omitted here for brevity.

Correctness follows since the evaluation of the ADP on that tensor $\mathbf{M}$ has the following property

$$\left(\mathbf{T} \cdot \left(\sum_{\ell=0}^{2n} x_\ell \mathbf{M}_\ell\right)\right)_{\cdot,j} = \mathbf{T} \cdot \left(\sum_{\ell=0}^{2n} x_\ell \mathbf{M}_\ell\right)_{\cdot,j} = \mathbf{T}\mathbf{C}_j\mathbf{x} = \mathbf{N}_j\mathbf{x},$$

so by just looking at the j-th column while peeling off the masking by $\mathbf{T}^{-1}$, we recover the application of the original NSS matrix slice $\mathbf{N}_j$. Since we showed before that a binary vector $\mathbf{x} \in \{0,1\}^k$ where $x_0 = 1$ satisfies $\mathbf{x}^\top \mathbf{N}_j \mathbf{x} = 0$, we know that $\mathbf{x}^\top \mathbf{T}$ is in the left kernel of $\mathbf{M}(\mathbf{x})$, proving that its rank drops on binary vectors, i.e., the distinguishing property is quadratic and defined by

$$\mathbf{x}^\top \mathbf{T}\mathbf{M}(\mathbf{x}) = \mathbf{0}^\top, \tag{2}$$

which is a vanishing hidden set of k quadratic equations on $\mathbf{x}$. From the perspective of an adversary, the only secret part of this is $\mathbf{T}$. Therefore, the heuristic security argument in [BIJ+20] relies on $\mathbf{T}$ staying hidden. In [BIJ+20, Section 7.2], the authors argue that linearization could be used to recover $\mathbf{T}$, yet they implicitly hint that if the VSS program is dense enough it would hide the AA program and make it difficult to recover.

In Appendix A, we describe a general black-box linearization attack on a certain generic template to build WE for NP from SNARKs for NP. We explain why rank-based constructions in [SRG+26; BIJ+20] are not immediately broken by this attack in Remark 3. However, the linearization attack on NSS-based ADP in the next section is a white-box analog of the black-box techniques in Appendix A, and we provide a full description in the appendix to motivate why the NSS-based construction is susceptible to this attack strategy while the formula-based constructions in Sections 2.3 and 2.4 are not.

In the next section, we show a concrete attack that recovers the randomness and the message, rendering this variant broken. We also provide an implementation and a summary of the results in Section 6.3.

4.2 Linearization Attack on NSS-Based ADP-Based WE

The attack is similar to the general template in Appendix A, except it is a white-box attack and not exactly black-box like Lemma 1. As explained in Remark 3, the rank-based construction is supposed to make the degree of the hidden polynomial prohibitively large to avoid this class of attacks. However, in the NSS-based AA ADP construction, despite remaining rank-based, the vanishing hidden polynomial is instead of low degree. That is because the rank drop depends on Eq. (2)

whenever $\mathbf{x}$ is binary [BIJ+20], and the hidden polynomial is only quadratic in $\mathbf{x}$.

Also, notice that a similar elimination for the linear VSS constraints can be applied as in Section 3, so we only need to consider bit checks applied in coordinates of the linear subspace of VSS solutions over $\mathbb{F}_p$.

Notice that Eq. (2) is a hidden affine polynomial in the entries of the random matrix $\mathbf{T}$, where the coefficients are monomials in the entries of $\mathbf{x}$ of degree at most 2. Since the encrypted message appears in the first matrix slice of the NSS variant, the remaining $k(k-1)$ entries of $\mathbf{T}_{1:k-1}$, the submatrix of $\mathbf{T}$ where the first row is removed, are completely determined from the rest of the slices, ignoring the slice at x_0.

Rather than working directly in the original $\mathbf{x}$-coordinates, we translate the problem to the modified program after the VSS program is absorbed. Hence, we get instead $\mathbf{z}^\top \widetilde{\mathbf{T}}\mathbf{M}(\mathbf{P}\mathbf{z})$ where $\widetilde{\mathbf{T}} \coloneqq \mathbf{P}^\top \mathbf{T}$ after substituting $\mathbf{x} = \mathbf{P}\mathbf{z}$. The attack then proceeds by recovering (most of) the entries of $\widetilde{\mathbf{T}}$. This recovery is unique up to scale, but it is not sufficient to recover all of $\mathbf{T}$ since $\mathbf{P}$, despite being public, is rectangular and $\widetilde{\mathbf{T}}$ is a (lossy) compression of $\mathbf{T}$. Recovery of $\widetilde{\mathbf{T}}$ is, however, sufficient in this setting to distinguish the encrypted bit b. Concretely, we will consider errors of the form $x_i^2 - x_0 x_i$ both for affine inputs and inputs at infinity, and we map those errors to linear equations on the hidden entries of $\widetilde{\mathbf{T}}$.

The attack proceeds in two stages.

1. **Recovery of $\widetilde{\mathbf{T}}$ Except for the First Row.** Since parametrization preserves $z_0 = x_0$ (see Section 3), restricting quadratic errors to inputs at infinity, i.e., $z_0 = 0$, eliminates the $\mathbf{T}$-independent term $bz_0\mathbf{S}$ from the encrypted bit.
2. **(Attempted) Recovery of the First Row of $\widetilde{\mathbf{T}}$.** In the second stage, we attempt to recover the first row of $\widetilde{\mathbf{T}}$ using quadratic errors from affine inputs instead. This turns out to be sufficient to distinguish the encrypted bit b.

We start with the first stage of the attack. Consider first the attack in the standard $\mathbf{x}$-coordinates. An adversary $\mathcal{A}$ collects N random vectors $\mathbf{x}^{(r)} \in \mathbb{F}_p^{2n}$ in the linear subspace of VSS solutions. Those vectors are intended to be solutions at infinity, and therefore $\mathcal{A}$ fixes $x_0 = 0$ and omits it from sampling. Then, $\mathcal{A}$ builds the matrix

$$\mathbf{Q} \coloneqq \left(\mathbf{q}(\mathbf{x}^{(0)}) \cdots \mathbf{q}(\mathbf{x}^{(N-1)})\right) \in \mathbb{F}_p^{2n \times N},$$

where $\mathbf{q}(\mathbf{x}) = \mathbf{x} \circ \mathbf{x}$. If $N > 2n$, $\ker(\mathbf{Q})$ is non-trivial. In fact, $\dim(\ker(\mathbf{Q})) \geq \max\{0, N - 2n\}$. Let $\alpha^{(s)} \in \ker(\mathbf{Q}) \subseteq \mathbb{F}_p^N$, and we know that given $x_0 = 0$ and

Eq. (1)

$$\sum_{r=0}^{N-1} \alpha_r^{(s)} \Big(\mathbf{x}^{(r)}\Big)^\top \mathbf{N}_j \mathbf{x}^{(r)} = \sum_{i=1}^{2n} \mathbf{N}_{j,i,i} \left[\sum_{r=0}^{N-1} \alpha_r^{(s)} \Big(x_i^{(r)}\Big)^2\right]$$
$$= \sum_{i=1}^{2n} \mathbf{N}_{j,i,i} \underbrace{\left[\sum_{r=0}^{N-1} \alpha_r^{(s)} q_i\Big(\mathbf{x}^{(r)}\Big)\right]}_{=0} = 0,$$

where $q_i(\mathbf{x}^{(r)})$ is the i-th coordinate of $\mathbf{q}(\mathbf{x}^{(r)})$. Given this,

$$\left[\sum_{r=0}^{N-1} \alpha_r^{(s)} \Big(\mathbf{x}^{(r)}\Big)^\top \mathbf{T}\mathbf{M}\Big(\mathbf{x}^{(r)}\Big)\right]_j = \sum_{r=0}^{N-1} \alpha_r^{(s)} \Big(\mathbf{x}^{(r)}\Big)^\top \mathbf{N}_j \mathbf{x}^{(r)} = 0$$
$$\implies \sum_{r=0}^{N-1} \alpha_r^{(s)} \Big(\mathbf{x}^{(r)}\Big)^\top \mathbf{T}\mathbf{M}\Big(\mathbf{x}^{(r)}\Big) = \mathbf{0}^\top.$$

Now, we consider the same attack in the $\mathbf{z}$-coordinates after the VSS program is absorbed. This has the advantage that we only sample vectors in $\mathrm{im}(\mathbf{P})$, which automatically satisfy the VSS part, isolating the NSS-based AA part to attack. The matrix $\widetilde{\mathbf{Q}} \in \mathbb{F}_p^{n \times N}$ has columns built from $\widetilde{\mathbf{q}}(\mathbf{z}) \coloneqq \mathbf{q}(\mathbf{P}\mathbf{z})$, where we only keep the first n rows of $\mathbf{Q}$. The reason for this is that the second half of the rows is redundant because

$$(x_0 - x_i)^2 - x_0(x_0 - x_i) = x_i^2 - x_0 x_i,$$

which is to say an input x_i is a binary bit if and only if its complement $x_0 - x_i$ is. The equality above proves this is true for affine inputs and inputs at infinity alike regardless of x_0. So we get instead the equations

$$\sum_{r=0}^{N-1} \alpha_r^{(s)} \Big(\mathbf{z}^{(r)}\Big)^\top \widetilde{\mathbf{T}}\widetilde{\mathbf{M}}\Big(\mathbf{z}^{(r)}\Big) = \mathbf{0}^\top \in \mathbb{F}_p^k. \tag{3}$$

Each $\alpha^{(s)}$ enforces k constraints on $\widetilde{\mathbf{T}}_{1:n-1}$, so we need $n-1$ independent $\alpha^{(s)}$ vectors. In fact, we could write concretely

$$\underbrace{\left(\sum_{r=0}^{N-1} \alpha_r^{(s)} \left[\widetilde{\mathbf{M}}\Big(\mathbf{z}^{(r)}\Big) \otimes \mathbf{z}^{(r)}\right]\right)^\top}_{\in \mathbb{F}_p^{k \times (n-1)k}} \mathrm{vec}(\widetilde{\mathbf{T}}_{1:n-1}) = \mathbf{0} \in \mathbb{F}_p^k, \tag{4}$$

to see that each kernel vector $\alpha^{(s)}$ contributes k equations on the vectorization of $\widetilde{\mathbf{T}}_{1:n-1}$.[10] Since that vectorization has length $k(n-1)$, this forces $\dim(\ker(\widetilde{\mathbf{Q}})) \geq n-1$, which in turn suggests that generically $N \geq n-1+n = 2n-1$.

Since the vectors $\alpha^{(s)}$ depend only on $\widetilde{\mathbf{Q}}$ and $\mathbf{z}$, they are expected to be independent of $\widetilde{\mathbf{T}}$ and to be sufficient to determine its coefficients up to scale (except for the first row). Experiments in Section 6.3 show that this is indeed the case and that the attack remains successful in all our tested instances.

For the second stage, we attempt to recover the first row of $\widetilde{\mathbf{T}}$. It turns out that we do not actually need to recover it and only need to check whether we can. This acts as an efficient distinguisher for the encrypted bit b. The procedure is similar to the first stage, except we now consider affine vectors with $z_0 \neq 0$. The errors instead become $x_i^2 - x_0 x_i$, and we only need $n+2$ columns in $\widetilde{\mathbf{Q}}$ since we only need the kernel to be at least 2-dimensional.

We take two different vectors α' and α'' in the kernel and we use the same method to try to recover the first row. If $b = 0$, we consistently get the same first row independent of the choice of α (up to the same scaling freedom as in the first stage). If $b = 1$, we get α-dependent answers for the first row. To increase the probability of success, we can increase the number of independent α vectors by increasing the size of the kernel, i.e., by increasing the number of sampled affine vectors in the second stage. However, our experiments indicate that the probability of failure is so low that two vectors were always sufficient to distinguish the encrypted bit b in all tested instances.

The attack complexity is dominated by solving a system of linear equations of size $\Theta(n^2)$ for the recovery step in the first stage. This amounts to the attack cost being $\mathcal{O}(n^{2\omega})$ field operations for a linear algebra constant $\omega \in [2,3]$, and for moderate n it is typically $\mathcal{O}(n^6)$. This means it is not as efficient as other attacks in this paper, but it still runs in $\mathcal{O}(\text{poly}(n))$. We refer the interested reader to Section 6.3 for implementation details, optimizations, and experimental results.

5 Commutator Attacks

We start by writing the matrices in Sections 2.3 and 2.4 for a circuit $\mathcal{C}$, but in a different format. Write the public tensor $\mathbf{M}$ as an order-3 tensor of size $N \times k \times k$ where $N = n+1$ for AADP and $N = 2n+1$ for ADP is the number of projective

[10] If we expand Eq. (3) to find the coefficients $\widetilde{\mathbf{T}}_{\ell,i}$ in the linear equation at index (j,s) for $j \in \{0,\ldots,k-1\}$ and $s \in \{0,\ldots,n-2\}$, we get

$$\sum_{\ell=1}^{n-1}\sum_{i=0}^{k-1} \underbrace{\left[\sum_{r=0}^{N-1} \alpha_r^{(s)} \left(\widetilde{\mathbf{M}}(\mathbf{z}^{(r)})_{i,j} z_\ell^{(r)}\right)\right]}_{(j,s)\text{-th equation coefficient}} \widetilde{\mathbf{T}}_{\ell,i} = 0,$$

so there are $k(n-1)$ independent equations (one for each column j of $\widetilde{\mathbf{M}}(\mathbf{z}^{(r)}) \otimes \mathbf{z}^{(r)} \in \mathbb{F}_p^{kn \times k}$ and kernel vector $\alpha^{(s)}$). To make them into row equations a final transpose is applied after tensoring in Eq. (4). All equations at $\ell = 0$ are dropped during the first stage since they are trivial when $z_0^{(r)} = 0$.

inputs, and $k = 2m + 1$ for AADP and $k = 2n^{1+\varepsilon} + 1$ for ADP is the program width. As noted in Section 3, we will treat the ADP construction as a special case of AADP, so we will focus on AADP in this section.

We know that $\mathbf{M}_i =: \sum_{j=0}^{m-1} \mathbf{L}_j \mathbf{U}_j^{(i)} \mathbf{R}_j$, where $\mathbf{L}_j, \mathbf{R}_j^\top \leftarrow \mathbb{F}_p^{k\times 4}$ and $\mathbf{U}_j^{(i)}$ is a local coefficient matrix for the i-th input and j-th gate. Notice that $\mathbf{U}_j(\mathbf{x}) =: \sum_{i=0}^{n} x_i \mathbf{U}_j^{(i)}$. This can be rewritten as $\mathbf{M}_i =: \mathbf{L}\mathbf{U}^{(i)}\mathbf{R}$, where $\mathbf{L} \coloneqq [\mathbf{L}_0 \mid \cdots \mid \mathbf{L}_{m-1}] \in \mathbb{F}_p^{k\times 4m}$, $\mathbf{R} \coloneqq [\mathbf{R}_0^\top \mid \cdots \mid \mathbf{R}_{m-1}^\top]^\top \in \mathbb{F}_p^{4m\times k}$, and $\mathbf{U}^{(i)} \coloneqq \mathrm{diag}(\mathbf{U}_0^{(i)}, \ldots, \mathbf{U}_{m-1}^{(i)})$. We split each $\mathbf{U}_j^{(i)} =: \mathbf{G}_j^{(i)} + \mathbf{\Xi}_j^{(i)}$ where $\mathbf{G}_j^{(i)} \in \mathbb{F}_p^{4\times 4}$ is the public coefficient part defined by $\mathcal{C}$ and $\mathbf{\Xi}_j^{(i)} \in \mathbb{F}_p^{4\times 4}$ is the secret masking part defined by $\mathbf{\Xi}$.

Now, rearrange the matrices so that $\mathbf{M}_i = \mathbf{L}^{(1)}\mathbf{\Xi}^{(i)}\mathbf{R}^{(2)} + \mathbf{L}\mathbf{G}^{(i)}\mathbf{R}$ where $\mathbf{L}^{(1)} \coloneqq [\mathbf{L}_{0,\cdot,0:1} \mid \cdots \mid \mathbf{L}_{m-1,\cdot,0:1}] \in \mathbb{F}_p^{k\times 2m}$, i.e., the collection of the first two columns of each $\mathbf{L}_j$, and similarly $\mathbf{L}^{(2)}$ is the collection of the last two columns, while $\mathbf{R}^{(1)}$ and $\mathbf{R}^{(2)}$ are the collections of the first two and last two rows of each $\mathbf{R}_j$, respectively. To see this, just consider the explicit split of $\mathbf{U}_j^{(i)}$

$$\underbrace{\begin{pmatrix} \mathbf{A}_{j,i} & \mathbf{C}_{j,i} & -\mathbf{\Xi}_{j,i} & 0 \\ \mathbf{D}_{j,i} & \mathbf{B}_{j,i} & 0 & \mathbf{\Xi}_{j,i} \\ 0 & 0 & \mathbf{B}_{j,i} & \mathbf{C}_{j,i} \\ 0 & 0 & \mathbf{D}_{j,i} & \mathbf{A}_{j,i} \end{pmatrix}}_{\mathbf{U}_j^{(i)}} = \underbrace{\begin{pmatrix} \mathbf{A}_{j,i} & \mathbf{C}_{j,i} & 0 & 0 \\ \mathbf{D}_{j,i} & \mathbf{B}_{j,i} & 0 & 0 \\ 0 & 0 & \mathbf{B}_{j,i} & \mathbf{C}_{j,i} \\ 0 & 0 & \mathbf{D}_{j,i} & \mathbf{A}_{j,i} \end{pmatrix}}_{\mathbf{G}_j^{(i)}} + \underbrace{\begin{pmatrix} 0 & 0 & -\mathbf{\Xi}_{j,i} & 0 \\ 0 & 0 & 0 & \mathbf{\Xi}_{j,i} \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{pmatrix}}_{\mathbf{\Xi}_j^{(i)}}.$$

The matrices $\mathbf{G}^{(i)}$ and $\mathbf{\Xi}^{(i)}$ are block diagonal. In every block, the matrix $\mathbf{\Xi}_j^{(i)}$ is supported only on its first two rows and last two columns. We therefore delete the identically zero rows and columns of $\mathbf{\Xi}^{(i)}$, together with the corresponding columns of $\mathbf{L}$ and rows of $\mathbf{R}$. This yields the reduced factorization $\mathbf{L}^{(1)}\mathbf{\Xi}^{(i)}\mathbf{R}^{(2)}$, where $\mathbf{L}^{(1)}$ collects the first two columns of every left-blinding matrix $\mathbf{L}_j$, and $\mathbf{R}^{(2)}$ collects the last two rows of every right-blinding matrix $\mathbf{R}_j$.

There are two important observations to make about this decomposition.

- The matrices $\mathbf{\Xi}^{(i)} = \mathrm{diag}(-\mathbf{\Xi}_{0,i}, \mathbf{\Xi}_{0,i}, \cdots, -\mathbf{\Xi}_{m-1,i}, \mathbf{\Xi}_{m-1,i}) \in \mathbb{F}_p^{2m\times 2m}$ are in particular all diagonal (hence commute) and are diagonalizable in the same basis defined by the $\mathbf{L}^{(1)}$ and $\mathbf{R}^{(2)}$ randomizations. They are also invertible except with negligible probability, and $\left(\mathbf{\Xi}^{(i)}\right)^{-1} = \mathrm{diag}(-\mathbf{\Xi}_{0,i}^{-1}, \mathbf{\Xi}_{0,i}^{-1}, \ldots, -\mathbf{\Xi}_{m-1,i}^{-1}, \mathbf{\Xi}_{m-1,i}^{-1})$ where inversions are (scalar) field inversions in $\mathbb{F}_p^\times$.
- Each matrix $\mathbf{G}^{(i)}$ has at most $\deg(i)$ nonzero blocks $\mathbf{G}_j^{(i)}$ for $j \in \mathrm{supp}(i)$ and each nonzero block has rank at most 4, so $\mathrm{rank}(\mathbf{G}^{(i)}) \le 4\deg(i)$.

Consider invertible matrices $\mathbf{L}', \mathbf{R}' \in \mathbb{F}_p^{k\times k}$ such that $\mathbf{L}'\mathbf{L}^{(1)} = \mathbf{I}'$ and $\mathbf{R}^{(2)}\mathbf{R}' = (\mathbf{I}')^\top$, where

$$\mathbf{I}' \coloneqq \begin{pmatrix} \mathbf{0}^\top \\ \mathbf{I}_{2m} \end{pmatrix}.$$

These matrices are uniquely determined by secret setup parameters. They are in fact specific approximate one-sided inverses of $\mathbf{L}^{(1)}$ and $\mathbf{R}^{(2)}$. Therefore,

$$\mathbf{L}'\mathbf{M}_i\mathbf{R}' = \begin{pmatrix} 0 & \mathbf{0}^\top \\ \mathbf{0} & \boldsymbol{\Xi}^{(i)} \end{pmatrix} + \mathbf{L}'\mathbf{L}\mathbf{G}^{(i)}\mathbf{R}\mathbf{R}'.$$

The first summand is singular, which is inconvenient because we will need to invert it below, so we add the matrix $\mathbf{e}_0\mathbf{e}_0^\top$ of rank one to it and subtract the same matrix from the second summand, setting

$$\mathbf{D}_i \coloneqq \begin{pmatrix} 1 & \mathbf{0}^\top \\ \mathbf{0} & \boldsymbol{\Xi}^{(i)} \end{pmatrix}, \quad \mathbf{E}_i \coloneqq \mathbf{L}'\mathbf{L}\mathbf{G}^{(i)}\mathbf{R}\mathbf{R}' - \mathbf{e}_0\mathbf{e}_0^\top.$$

With this choice, $\mathbf{L}'\mathbf{M}_i\mathbf{R}' = \mathbf{D}_i + \mathbf{E}_i$ holds exactly, where $\mathbf{D}_i$ is diagonal and invertible and $\mathbf{E}_i$ is a low-rank error with $\operatorname{rank}(\mathbf{E}_i) \leq 4\deg(i) + 1$, the extra 1 being the contribution of $\mathbf{e}_0\mathbf{e}_0^\top$. In other words, this is an almost-diagonal matrix up to a low-rank error. Since $\mathbf{L}', \mathbf{R}'$ are invertible, we will henceforth directly treat $\mathbf{M}_i = \mathbf{D}_i + \mathbf{E}_i$ as they have exactly the same rank profiles.

For a sparse circuit, the i-th variable appears in only a few gates, so $\deg(i) = |\operatorname{supp}(i)|$ is small. Consequently, the errors $\mathbf{E}_i$ have very low rank, which allows us to define an efficient distinguisher based on Strassen commutator relations [Str83] for tensor rank, as we explain shortly. In [BIJ+20], the situation reaches the extreme case where $\deg(i) = 1$ for $i \in \{0, \ldots, n\}$ and the rank of the error term is at most 5, except at $i = 0$, because variables occur in only one bit-checking block.

Let $\mathbf{T}^L_{a;b} \coloneqq \mathbf{M}_a^{-1}\mathbf{M}_b$ be the left quotient at wire indices $a, b \in [n]$ and similarly $\mathbf{T}^R_{a;b} \coloneqq \mathbf{M}_b\mathbf{M}_a^{-1}$ be the corresponding right quotient. Consider the commutators $\mathsf{C}_L(a; b, c) \coloneqq [\mathbf{T}^L_{a;b}, \mathbf{T}^L_{a;c}]$ and $\mathsf{C}_R(a; b, c) \coloneqq [\mathbf{T}^R_{a;b}, \mathbf{T}^R_{a;c}]$.[11] Moving forward, we will consider the left quotient commutators to explain the distinguisher, and we will show parallels for the right quotient commutators whenever necessary.

The reason we work with commutators of quotients instead of commutators of the public matrices themselves is only clear if we look at the actual public matrices, which have independent left and right randomizations, so if we look at their commutators with randomizations included

$$[\mathbf{L}'\mathbf{M}_a\mathbf{R}', \mathbf{L}'\mathbf{M}_b\mathbf{R}'] = \mathbf{L}'\mathbf{M}_a\mathbf{R}'\mathbf{L}'\mathbf{M}_b\mathbf{R}' - \mathbf{L}'\mathbf{M}_b\mathbf{R}'\mathbf{L}'\mathbf{M}_a\mathbf{R}',$$

we get some internal pollution from $\mathbf{R}'\mathbf{L}'$, but if we consider the quotients, they turn out to be conjugations instead, i.e.,

$$\mathbf{T}^L_{a;b} = (\mathbf{R}')^{-1}\mathbf{M}_a^{-1}\underbrace{(\mathbf{L}')^{-1}\mathbf{L}'}_{\mathbf{I}_k}\mathbf{M}_b\mathbf{R}' = (\mathbf{R}')^{-1}\mathbf{M}_a^{-1}\mathbf{M}_b\mathbf{R}',$$

[11] The latter is typically the one considered in Strassen commutator relations.

and, similarly, $\mathbf{T}^R_{a;b} = \mathbf{L}'\mathbf{M}_b\mathbf{M}_a^{-1}(\mathbf{L}')^{-1}$. Commutators are then invariant to common conjugation because

$$\begin{aligned}[\mathbf{Z}^{-1}\mathbf{X}\mathbf{Z}, \mathbf{Z}^{-1}\mathbf{Y}\mathbf{Z}] &= \mathbf{Z}^{-1}\mathbf{X}\underbrace{\mathbf{Z}\mathbf{Z}^{-1}}_{\mathbf{I}}\mathbf{Y}\mathbf{Z} - \mathbf{Z}^{-1}\mathbf{Y}\underbrace{\mathbf{Z}\mathbf{Z}^{-1}}_{\mathbf{I}}\mathbf{X}\mathbf{Z} \\ &= \mathbf{Z}^{-1}(\mathbf{X}\mathbf{Y} - \mathbf{Y}\mathbf{X})\mathbf{Z} \\ &= \mathbf{Z}^{-1}[\mathbf{X}, \mathbf{Y}]\mathbf{Z},\end{aligned}$$

and that justifies ignoring the common conjugation in the argument that follows. In other words, we can simply drop the random setup-dependent gauge.

First, notice that

$$\begin{aligned}&\mathbf{M}_a^{-1}(\mathbf{D}_a + \mathbf{E}_a) = \mathbf{I}_k \\ &\implies \mathbf{M}_a^{-1}\mathbf{D}_a = \mathbf{I}_k - \mathbf{M}_a^{-1}\mathbf{E}_a \\ &\implies \mathbf{M}_a^{-1} = \mathbf{D}_a^{-1} - \mathbf{M}_a^{-1}\mathbf{E}_a\mathbf{D}_a^{-1}.\end{aligned}$$

Next, the left quotient is

$$\begin{aligned}\mathbf{T}^L_{a;b} &= \underbrace{(\mathbf{D}_a^{-1} - \mathbf{M}_a^{-1}\mathbf{E}_a\mathbf{D}_a^{-1})}_{=\mathbf{M}_a^{-1}}(\mathbf{D}_b + \mathbf{E}_b) \\ &= \underbrace{\mathbf{D}_a^{-1}\mathbf{D}_b}_{\mathbf{\Lambda}_{a;b}} + \underbrace{\mathbf{M}_a^{-1}\mathbf{E}_b - \mathbf{M}_a^{-1}\mathbf{E}_a\mathbf{D}_a^{-1}\mathbf{D}_b}_{\mathbf{E}_{a;b}},\end{aligned}$$

where $\mathbf{\Lambda}_{a;b}$ is diagonal and $\operatorname{rank}(\mathbf{E}_{a;b}) \leq \operatorname{rank}(\mathbf{E}_a) + \operatorname{rank}(\mathbf{E}_b) = \mathcal{O}(\deg(a) + \deg(b))$.[12] Next, consider the left quotient commutator, and since commutators are bilinear, we get

$$\begin{aligned}\mathsf{C}_L(a; b, c) &= [\mathbf{\Lambda}_{a;b} + \mathbf{E}_{a;b}, \underbrace{\mathbf{\Lambda}_{a;c} + \mathbf{E}_{a;c}}_{=\mathbf{T}^L_{a;c}}] \\ &= \underbrace{[\mathbf{\Lambda}_{a;b}, \mathbf{\Lambda}_{a;c}]}_{=0} + [\mathbf{E}_{a;b}, \mathbf{T}^L_{a;c}] + [\mathbf{\Lambda}_{a;b}, \mathbf{E}_{a;c}],\end{aligned}$$

where we use the fact that diagonal matrices commute to cancel the first term. Finally, we use the simple fact that $\operatorname{rank}([\mathbf{A}, \mathbf{B}]) \leq 2\min\{\operatorname{rank}(\mathbf{A}), \operatorname{rank}(\mathbf{B})\}$ to deduce that $\operatorname{rank}(\mathsf{C}_L(a; b, c)) \leq \mathcal{O}(\deg(a)+\deg(b)+\deg(c))$.[13] Similar arguments can be applied to right quotient commutators to deduce that $\operatorname{rank}(\mathsf{C}_R(a; b, c)) \leq \mathcal{O}(\deg(a) + \deg(b) + \deg(c))$.

A typical circuit has most of its wires appearing in very few gates, which makes the distinguisher even stronger, so that it can distinguish several such index triples from random. The choice of indices for a concrete exploit of the distinguisher is, however, clearly circuit-dependent.

[12] The error can in fact be written as $\mathbf{M}_a^{-1}(\mathbf{E}_b - \mathbf{E}_a\mathbf{\Lambda}_{a;b})$ and since $\mathbf{M}_a^{-1}$ is invertible we only need the rank of $\mathbf{E}_b - \mathbf{E}_a\mathbf{\Lambda}_{a;b}$. Additionally, since $\mathbf{\Lambda}_{a;b}$ is diagonal with first diagonal entry exactly 1, we get that $\mathbf{e}_0\mathbf{e}_0^\top\mathbf{\Lambda}_{a;b} = \mathbf{e}_0\mathbf{e}_0^\top$ and it cancels with the same rank-1 error in $\mathbf{E}_b$, so the rank cannot exceed $4(\deg(a) + \deg(b))$. We only need the asymptotic bound on sparse circuits though to describe the distinguisher.

[13] Building on Footnote 12, a concrete bound on that rank is $4(2\deg(a) + \deg(b) + \deg(c))$.

5.1 Discovering Secrets

The distinguisher introduced above may subsequently be used to find vectors $\mathbf{u} \in \ker((\mathbf{L}^{(1)})^\top)$ and $\mathbf{v} \in \ker(\mathbf{R}^{(2)})$ which must exist by mere dimensional arguments. Indeed, $\mathbf{R}^{(2)} \in \mathbb{F}_p^{2m\times k}$ has full row rank $2m = k-1$ except with negligible probability, so its kernel is one-dimensional and $\mathbf{v}$ is unique up to a nonzero scalar, and symmetrically for $(\mathbf{L}^{(1)})^\top$ and $\mathbf{u}$.

Such a pair is all an attacker needs. Grouping the columns of $\mathbf{L}$ as $[\mathbf{L}^{(1)} \mid \mathbf{L}^{(2)}]$ and the rows of $\mathbf{R}$ accordingly, the identically zero bottom-left block of every $\mathbf{U}_j^{(i)}$ leaves

$$\mathbf{M}_i = \mathbf{L}^{(1)}(\cdots)\mathbf{R}^{(1)} + \mathbf{L}^{(1)}\boldsymbol{\Xi}^{(i)}\mathbf{R}^{(2)} + \mathbf{L}^{(2)}(\cdots)\mathbf{R}^{(2)},$$

so every term starts with $\mathbf{L}^{(1)}$ or ends with $\mathbf{R}^{(2)}$, and hence $\langle \mathbf{u}, \mathbf{M}_i\mathbf{v}\rangle = 0$ for all i. The message escapes this only because it is added to the bottom-right entry of the constant matrix rather than through a gate, so

$$\big\langle \mathbf{u}, \big(\mathbf{M}_0 + \mathtt{msg}\,\mathbf{e}_{k-1}\mathbf{e}_{k-1}^\top\big)\mathbf{v}\big\rangle = \mathtt{msg}\cdot u_{k-1}v_{k-1},$$

and dividing by $u_{k-1}v_{k-1}$, which is nonzero except with negligible probability, returns $\mathtt{msg}$. Both conditions are computable from the ciphertext alone, so a candidate pair is certified by testing $\langle \mathbf{u}, \mathbf{M}_i\mathbf{v}\rangle = 0$ for all $i \geq 1$ together with $u_{k-1}v_{k-1} \neq 0$.

The distinguisher from above yields those two vectors. First, note that it was invariant under the gauge $\mathbf{L}', \mathbf{R}'$ and could therefore be argued in the normalized frame $\mathbf{M}_i = \mathbf{D}_i + \mathbf{E}_i$. The recovery targets that gauge itself, so from here on we work with the public matrices as they are, i.e.,

$$\mathbf{M}_i = \mathbf{L}^{(1)}\boldsymbol{\Xi}^{(i)}\mathbf{R}^{(2)} + \mathbf{F}_i, \quad \mathbf{F}_i \coloneqq \mathbf{L}\mathbf{G}^{(i)}\mathbf{R}, \quad \text{where } \operatorname{rank}(\mathbf{F}_i) \leq 4\deg(i).$$

We write $\operatorname{row}(\mathbf{X})$ for the span of the rows of $\mathbf{X}$ and $\operatorname{im}(\mathbf{X})$ for the span of its columns. In the following, we make the relation between the ultimate goal of finding $\mathbf{v}$ (and $\mathbf{u}$) and the commutators described above explicit.

Relation Between $\mathbf{v}$ and $\mathbf{R}^{(2)}$. Reading $\mathbf{R}^{(2)}\mathbf{v} = \mathbf{0}$ row-wise shows that every row of $\mathbf{R}^{(2)}$ annihilates $\mathbf{v}$, so $\operatorname{row}(\mathbf{R}^{(2)})$ is contained in the hyperplane of vectors orthogonal to $\mathbf{v}$. Both spaces have dimension $k-1$, hence

$$\operatorname{row}(\mathbf{R}^{(2)}) = \big\{\mathbf{r} \in \mathbb{F}_p^k \;\big|\; \langle \mathbf{r}, \mathbf{v}\rangle = 0\big\},$$

and so a vector lies in the row space of the secret $\mathbf{R}^{(2)}$ if and only if it annihilates $\mathbf{v}$. Any $2m$ linearly independent vectors known to lie in $\operatorname{row}(\mathbf{R}^{(2)})$ therefore span all of it, and stacking them as the rows of a matrix leaves precisely the multiples of $\mathbf{v}$ in its right kernel. We emphasize that $\mathbf{R}^{(2)}$ having a corank of 1 is crucial here, since otherwise, e.g., at $\operatorname{rank}(\mathbf{R}^{(2)}) = 2m-1$, that kernel would be two-dimensional.

Relation Between $\mathbf{R}^{(2)}$ and Commutators. The rows of a product $\mathbf{XY}$ are combinations of the rows of $\mathbf{Y}$, so $\operatorname{row}(\mathbf{XY}) \subseteq \operatorname{row}(\mathbf{Y})$ and only the rightmost factor of a product matters here. Abbreviating $\mathbf{S}_x \coloneqq \mathbf{M}_a^{-1}\mathbf{M}_x\mathbf{M}_a^{-1}$ and

suppressing the fixed base a, the left quotient commutator expands to

$$\begin{aligned}\mathsf{C}_L(a;b,c) &= (\mathbf{M}_a^{-1}\mathbf{M}_b)(\mathbf{M}_a^{-1}\mathbf{M}_c) - (\mathbf{M}_a^{-1}\mathbf{M}_c)(\mathbf{M}_a^{-1}\mathbf{M}_b) = \mathbf{S}_b\mathbf{M}_c - \mathbf{S}_c\mathbf{M}_b \\ &= \underbrace{\Big(\mathbf{S}_b\mathbf{L}^{(1)}\boldsymbol{\Xi}^{(c)} - \mathbf{S}_c\mathbf{L}^{(1)}\boldsymbol{\Xi}^{(b)}\Big)}_{k\times 2m}\mathbf{R}^{(2)} + \mathbf{S}_b\mathbf{F}_c - \mathbf{S}_c\mathbf{F}_b.\end{aligned}$$

The first term has $\mathbf{R}^{(2)}$ as its rightmost factor, so all of its rows already lie in $\mathrm{row}(\mathbf{R}^{(2)})$. The other two end in $\mathbf{F}_c$ and $\mathbf{F}_b$ and hence contribute at most $\mathrm{rank}(\mathbf{F}_b) + \mathrm{rank}(\mathbf{F}_c) \le 4(\deg(b) + \deg(c))$ further directions, so

$$\mathrm{row}(\mathsf{C}_L(a;b,c)) \subseteq \mathrm{row}(\mathbf{R}^{(2)}) + \mathrm{row}(\mathbf{F}_b) + \mathrm{row}(\mathbf{F}_c).$$

In other words, the commutator sits inside the hyperplane we are after except for at most $4(\deg(b) + \deg(c))$ wrong directions, which is again small precisely when the circuit is sparse.

Remark 2. This is a claim about directions and not about individual rows, as generically every single row mixes a genuine part with some error and is unusable on its own. Moreover, the space containing the wrong directions depends only on b and c and not on the base a, since $\mathbf{S}_b$ and $\mathbf{S}_c$ are left factors and do not affect row spaces, so unlike the rank bound above it does not grow with $\deg(a)$.

Removing the Wrong Directions. We cannot intersect $\mathrm{row}(\mathsf{C}_L)$ with $\mathrm{row}(\mathbf{R}^{(2)})$ directly since $\mathbf{R}^{(2)}$ is secret, so we intersect two commutators against each other instead. Two commutators built from different index triples share $\mathbf{L}^{(1)}$ and $\mathbf{R}^{(2)}$, but the directions in which they actually escape $\mathrm{row}(\mathbf{R}^{(2)})$ come from $\mathbf{S}_b\mathbf{F}_c - \mathbf{S}_c\mathbf{F}_b$ and hence already differ when only the base a is changed, so keeping only what their row spaces have in common discards those and retains what is actually contained in $\mathrm{row}(\mathbf{R}^{(2)})$.

To see why this is meaningful, consider the following argument. For any two subspaces $U, W \subseteq \mathbb{F}_p^k$ we have $\dim(U \cap W) = \dim(U) + \dim(W) - \dim(U + W)$, and generic subspaces span as much as the ambient space allows, so they meet in dimension $\max\{0, \dim(U) + \dim(W) - k\}$. Whenever the two commutators have ranks summing to less than k, which is exactly what the distinguisher and the sparsity assumption give, generic subspace intersections would only yield $\{\mathbf{0}\}$. Any nonzero vector surviving the intersection is therefore evidence of a shared hidden structure, and the only structure the two commutators share is $\mathbf{L}^{(1)}$ and $\mathbf{R}^{(2)}$.

Collecting Sufficiently Many Rows. Repeating this over many index triples and stacking all surviving vectors as the rows of one matrix, its right kernel collapses to the multiples of $\mathbf{v}$ as soon as $2m$ linearly independent rows have been collected. The right quotient commutators give $\mathbf{u}$ by the transposed argument, with column spaces in place of row spaces, since there it is the leftmost rather than the rightmost factor that survives, i.e.,

$$\mathsf{C}_R(a;b,c) = \mathbf{M}_b\mathbf{S}_c - \mathbf{M}_c\mathbf{S}_b = \mathbf{L}^{(1)}\Big(\boldsymbol{\Xi}^{(b)}\mathbf{R}^{(2)}\mathbf{S}_c - \boldsymbol{\Xi}^{(c)}\mathbf{R}^{(2)}\mathbf{S}_b\Big) + \mathbf{F}_b\mathbf{S}_c - \mathbf{F}_c\mathbf{S}_b,$$

and hence $\mathrm{im}(\mathsf{C}_R(a;b,c)) \subseteq \mathrm{im}(\mathbf{L}^{(1)}) + \mathrm{im}(\mathbf{F}_b) + \mathrm{im}(\mathbf{F}_c)$.

Which triples are needed, and how many, is circuit-dependent. In our experiments, generic families such as consecutive triples $\mathsf{C}_L(a;a+1,a+2)$ and base swaps (i.e., intersecting $\mathsf{C}_L(a;a+1,a+2)$ with $\mathsf{C}_L(a+1;a,a+2)$) already yield most of the $2m$ directions on the circuits we implemented. The few remaining ones belong to gates whose public coefficients are proportional across the slices involved, and reaching them needs choices tailored to the circuit. There is also a size floor, since the recovery needs k to be comfortably larger than the per-slice error ranks so that the wrong directions the intersections are meant to cancel do not overlap by accident. Both points are discussed in Sections 6.1 and 6.2.

Once $\mathbf{u}$ and $\mathbf{v}$ are in hand, the message is read off as above, so the rank-based WE schemes in [SRG+26; BIJ+20] lose both soundness and extractability.

It is finally worth mentioning that the attack is dominated by public matrix inversions, matrix multiplications, and linear solvers to find kernel spaces, which all add up to $\mathcal{O}(N\,k^2) = \mathcal{O}(n^3)$ if we make the reasonable assumption that $m = \Theta(n)$. For a typical circuit, finding kernel intersections requires repeating this process for $\mathcal{O}(n)$ triples of indices, which brings the attack complexity to $\mathcal{O}(n^4)$. The attack is therefore polynomial-time and efficient to mount.

6 Practical Evaluation

In order to verify the observations of Section 5, we implemented the attack in `Sage` and used it against two targets, the original ADP witness encryption of [BIJ+20] and a native AADP circuit [SRG+26]. In both cases the attacker is given only the public ciphertext, that is, the matrices $\mathbf{M}_i$ together with the public circuit, and recovers the encrypted message without a witness and without any of the secret data $\mathbf{L}_j$, $\mathbf{R}_j$, and $\boldsymbol{\Xi}$. The commutator starting point of Section 5 is identical in the two cases. What differs is the circuit and with it the particular family of commutators that has to be combined in order to recover the kernels.

6.1 The Original ADP Construction

Attack Overview. The ciphertext is the family of per-variable matrices $\mathbf{M}_i$ of the subset sum construction. By the reduction of Section 3, this is equivalent to the bit-check circuit defined below, which is the form we attack. For a public instance $\langle(-\ell \mid \mathbf{h}), (1 \mid \mathbf{w})\rangle = 0$ we take the width-two basis $\mathbf{c}^{(i)} = h_{i+1}\mathbf{e}_i - h_i\mathbf{e}_{i+1}$ of the directions that leave the sum $\langle \mathbf{h}, \mathbf{w}\rangle$ unchanged and form

$$\widetilde{\mathbf{M}}_i = h_{i+1}\,\mathbf{M}_i - h_i\,\mathbf{M}_{i+1}, \qquad 1 \le i \le m-1,$$

a combination that uses only the public coefficients $\mathbf{h}$. Intuitively, $\mathbf{c}^{(i)}$ raises w_i by h_{i+1} and lowers w_{i+1} by h_i, and these two changes cancel in the weighted sum, so they leave $\langle \mathbf{h}, \mathbf{w}\rangle$ unchanged. These changes touch only two neighboring variables, which is what keeps the resulting matrices sparse. For the same reason, the matrices that enforce $\langle \mathbf{h}, \mathbf{w}\rangle = \ell$ enter every such combination weighted by $\langle \mathbf{h}, \mathbf{c}^{(i)}\rangle = 0$, so they cancel and leave only bit-checks. These combinations

change $\mathbf{w}$ without changing the sum, so to reach the target ℓ we still need one solution to anchor them. In our experiment, we use the binary solution $\mathbf{w}^\star$ of the instance, folded into the constant matrix as $\mathbf{M}_0 + \sum_i w_i^\star \mathbf{M}_i$, which makes the all-zero assignment $\mathbf{s} = \mathbf{0}$ a witness, so we can also confirm honest decryption. Here $\mathbf{M}_0$ is the matrix attached to the fixed input $w_0 = 1$ rather than to any variable. The attack itself never uses $\mathbf{w}^\star$. Together with the $m-1$ matrices $\widetilde{\mathbf{M}}_i$, this gives a bit-check circuit over the new variables s_i. Our implementation builds this reduced circuit directly, the cancellation of the subset sum part being precisely the content of that reduction.

On this circuit we recover the kernels as in Section 5. With the left quotients $\mathbf{T}^L_{a;b} = \widetilde{\mathbf{M}}_a^{-1}\widetilde{\mathbf{M}}_b$ taken on the converted matrices, the rows of $\mathsf{C}_L(a; b, c)$ lie in the row space of $\mathbf{R}^{(2)}$ up to a low-rank term. For this circuit, the combinations that cancel that term are exactly the triples of variable indices $\{a, b, b+1\}$ whose two larger indices are adjacent. For each such triple, we form the three commutators that take a, b and $b+1$ in turn as the base shared by the two quotients, and we intersect their row spaces. Ranging over all $a < b$, the collected rows span the full row space of $\mathbf{R}^{(2)}$ and pin down the one-dimensional right kernel $\mathbf{v} \in \ker(\mathbf{R}^{(2)})$. The vector $\mathbf{u} \in \ker((\mathbf{L}^{(1)})^\top)$ follows from the right quotient commutators C_R in the same way.

The genuine $\mathbf{u}$ and $\mathbf{v}$ are the left and right kernels of the message-free part of the construction, so they satisfy $\left\langle \mathbf{u}, \widetilde{\mathbf{M}}_i \, \mathbf{v} \right\rangle = 0$ for every $i \in \{1, \ldots, m-1\}$. Evaluating this identity on the public matrices lets the attacker confirm, from the ciphertext alone, that the two recovered vectors are the correct ones. The message is a single bit b. Following [BIJ+20], the encryptor adds it to the constant matrix as a further $b\,\mathbf{S}$ for a uniformly random $\mathbf{S}$. Since $\mathbf{u}$ and $\mathbf{v}$ annihilate the message-free part, applying $\langle \mathbf{u}, \cdot\, \mathbf{v} \rangle$ to the published constant matrix leaves only $b \cdot \langle \mathbf{u}, \mathbf{S}\,\mathbf{v} \rangle$, and the attacker reads $b = 1$ exactly when this value is nonzero. This bit is the message of the original ADP ciphertext, and we read it from the bit-check circuit with no witness, using only the public matrices.

Circuit Definition. The construction of [BIJ+20, Section 5.2] is a witness encryption for subset sum, where a witness is a binary $\mathbf{w} \in \{0,1\}^n$ with $\langle \mathbf{h}, \mathbf{w} \rangle = \ell$ for public $\mathbf{h}$ and ℓ. Finding such a $\mathbf{w}$ is NP-hard.

The reduction of Section 3 removes the equation by a change of variables. Its solutions form an affine subspace over $\mathbb{F}_p$, and writing the construction over a particular solution and a basis of its directions makes the equation hold by construction, so it disappears as an explicit constraint and the subset sum matrices that enforced it cancel. What remains is a pure bit-check circuit that encrypts the same bit. A witness for it is still a binary point of the subspace, that is, a subset sum solution, so breaking this bit-check circuit breaks the original scheme.

This reduction holds for any basis of those directions. In particular, we use the width-two basis above, which exists because the constraint is a single equation. A dense basis would instead give dense matrices in which the low-rank commutator structure likely never appears. That basis depends only on the public $\mathbf{h}$, so the

attacker forms it with no witness. Each new variable s_i then meets only the two neighboring gates $i-1$ and i, so $\deg(i) = 2$ and the circuit lies in the sparse regime of Section 5. The gates are the 2×2 ADP gates $\left(\begin{smallmatrix} \mathbf{A}_{j,i} & \mathbf{\Xi}_{j,i} \\ 0 & \mathbf{B}_{j,i} \end{smallmatrix}\right)$ whose masking part $\mathbf{\Xi}_j^{(i)}$ has rank one, so that $k = m + 1$.

6.2 The AADP Construction

Attack Overview. Here the public matrices $\mathbf{M}_i$ are attacked directly, with no conversion. We recover the kernels with the same commutators, and as with ADP, the consecutive triples $\{i, i+1, i+2\}$ and their base swaps already give most of the rows. Two places in the squaring chain need extra combinations, and we add one family for each. The first is the start of the chain. The consecutive rule needs a variable that has neighbors on both sides, and the earliest variables do not, so the rows of the first few gates never show up. A few combinations tailored to those early variables supply the missing rows. The second is the gate that combines the input bits into $\sum_t 2^t b_t$ before squaring. The bits enter this gate as fixed multiples 2^t of one another, so across this one gate their three matrices are proportional and the commutator cannot tell them apart. One row of $\mathbf{R}^{(2)}$ stays hidden as a result. We recover it by taking the single weighted combination of those matrices that cancels the gate. Intersecting all three families yields the row space of $\mathbf{R}^{(2)}$ and the column space of $\mathbf{L}^{(1)}$, and hence $\mathbf{v}$ and $\mathbf{u}$. The same check confirms $\mathbf{u}$ and $\mathbf{v}$ here, and since the message is added to the bottom-right entry of $\mathbf{M}_0$ we recover it as $\mathtt{msg} = \langle \mathbf{u}, \mathbf{M}_0\, \mathbf{v}\rangle / (u_{k-1} v_{k-1})$.

Circuit Definition. The AADP circuit is a squaring chain. Three bit-check gates force the input variables to be binary, a first squaring gate composes them into $x = \sum_t 2^t b_t$ and squares it, and the remaining gates iterate $y \mapsto y^2$, the last one checking the result against the public target. The AADP gates are 4×4 and the masking part $\mathbf{\Xi}_j^{(i)}$ has rank two, so that $k = 2m + 1$.

6.3 The NSS-Based Construction

Attack Overview. There are k public matrices, each of size $k \times k$, where $k = 2n + 1$. The full description of the matrices matches exactly the VSS ADP description in Section 2.3 and the NSS-based AA-ADP (all-accept ADP) in Section 4.1.

We first construct a matrix $\mathbf{P} \in \mathbb{F}_p^{k\times n}$ as in Section 3 to absorb the contribution of VSS and keep only the contribution of the NSS-based AA-ADP. We use $\mathbf{P}$ to build $\widetilde{\mathbf{M}}_\ell$ for $\ell \in [n]$ and then apply the attack to the new program instead. Recall that we ensure $z_0 = w_0$. The attack proceeds in two stages.

1. In the first stage, we sample $N_1 = 2n - 1$ vectors $\widehat{\mathbf{z}}^{(r)} \leftarrow \mathbb{F}_p^{n-1}$, so that they are valid inputs at infinity, with $\widehat{z}_0^{(r)} = 0$ fixed as the remaining coordinate. We define $\widetilde{\mathbf{P}}_\infty \coloneqq \mathbf{P}_{1:n,1:n-1} \in \mathbb{F}_p^{n\times(n-1)}$ to collect only the vectors $\mathbf{w} \in \mathbb{F}_p^n$, ignoring $w_0 = 0$, and their complements $-\mathbf{w}$ in the lower coordinates, then

(a) We form their error matrix $\widetilde{\mathbf{Q}}_\infty \in \mathbb{F}_p^{n \times N_1}$ where each column r is $\mathbf{q}_\infty(\mathbf{w}^{(r)}) = \mathbf{w}^{(r)} \circ \mathbf{w}^{(r)}$ for $\mathbf{w}^{(r)} = \widetilde{\mathbf{P}}_\infty \widehat{\mathbf{z}}^{(r)}$. This is basically $w_i^2 - w_0 w_i$ at $z_0 = w_0 = 0$ since the parametrization preserves $w_0 = z_0$.

(b) We find a basis for $\ker(\widetilde{\mathbf{Q}}_\infty)$ whose dimension is guaranteed to be at least $N_1 - n = n - 1$.

(c) Each vector creates k independent linear relations on $\widetilde{\mathbf{T}}_{1:n-1,\cdot}$, i.e., on the bottom $n-1$ rows of $\widetilde{\mathbf{T}}$ simultaneously, and $n-1$ of them are sufficient to recover $\widetilde{\mathbf{T}}_{1:n-1}$ up to scale. That requires the system of linear equations to have nullity 1, which is verified during execution.

2. In the second stage, we sample $N_2 = n + 2$ vectors $\mathbf{z}^{(r)} \leftarrow \mathbb{F}_p^n$, so that they are taken to be affine inputs, i.e., $z_0^{(r)} \neq 0$.[14] We attempt to recover the first row of $\widetilde{\mathbf{T}}$, but we use this recovery process as a distinguisher for the encrypted bit b. We also want to omit the complements $w_0\mathbf{1} - \mathbf{w}$ from the image of $\mathbf{P}$, so we define $\widetilde{\mathbf{P}}_{\text{aff}} \coloneqq \mathbf{P}_{0:n,0:n-1} \in \mathbb{F}_p^{(n+1)\times n}$.

(a) We again form their error matrix $\widetilde{\mathbf{Q}}_{\text{aff}} \in \mathbb{F}_p^{n \times N_2}$ where each column r is $\mathbf{q}_{\text{aff}}(\mathbf{w}^{(r)}) = \mathbf{w}^{(r)} \circ \mathbf{w}^{(r)} - w_0^{(r)}\mathbf{w}^{(r)}$ for $(w_0^{(r)} \mid \mathbf{w}^{(r)}) = \widetilde{\mathbf{P}}_{\text{aff}} \mathbf{z}^{(r)}$.

(b) We find a basis for $\ker(\widetilde{\mathbf{Q}}_{\text{aff}})$ and the dimension is at least 2.

(c) In the absence of the encrypted bit component $b\mathbf{S}$, a single vector $\alpha \in \ker(\widetilde{\mathbf{Q}}_{\text{aff}})$ suffices to recover the first row of $\widetilde{\mathbf{T}}$, but we sample at least two to distinguish the encrypted message bit b as follows.

- If $b = 0$, we always recover the same row up to the same scale as in the first stage.
- If $b = 1$, the recovered row depends on $\alpha \in \ker(\widetilde{\mathbf{Q}}_{\text{aff}})$ and $\mathbf{S}$, and since $\mathbf{S}$ and $\mathbf{T}$ are sampled independently, we generally get different answers.

Notice that recovering different first rows of $\widetilde{\mathbf{T}}$ in the second stage is a perfect distinguisher for $b = 1$, but when the answers match there is a low probability of failure to detect $b = 1$. The failure probability diminishes as the slack increases beyond 2, but in our experiments, the probability of failure is so low that *none* of the recovered first rows coincide even for a larger slack[15] and we never failed to distinguish $b = 1$.

Circuit Definition. The circuit is a subset sum instance where $\mathbf{h} = \mathbf{1} \in \mathbb{F}_p^n$ and $\ell = n + 1$, which is then embedded into a VSS instance exactly as in [BIJ+20, Section 5.2]. For $p > n + 1$, there are no binary solutions, so the subset sum instance is unsatisfiable, but there are solutions to the linear system over $\mathbb{F}_p$.

[14] We never need to resample vectors to ensure the affine condition since the probability of failure is $1/p = \text{negl}(\lambda)$.

[15] We tested 100 attempts with unique random seeds up to $N_2 = n + 50$ for $n = 8$ and verified that all 50 recovered rows are *unique*, which is stronger than the condition we need.

Table 1: Attack timings over the BN254 scalar field, single-threaded. The reported time covers the full attack on the public ciphertext, that is, recovery of $\mathbf{u}$ and $\mathbf{v}$, the check on them, and reading the message.

Scheme	Circuit	m	k	Time (s)
ADP [BIJ+20]	Subset sum bit-checks	16	17	3.9
		24	25	27.8
		32	33	115.2
AADP [SRG+26]	Squaring chain	16	33	5.1
		24	49	22.8
		32	65	68.8
		48	97	339.0

6.4 Practical Results

Both attacks in Sections 6.1 and 6.2 reduce to dense linear algebra over $\mathbb{F}_p$ on $k \times k$ matrices and run in polynomial time. Table 1 reports timings over the BN254 scalar field for growing instance sizes. The attack succeeds in every case. We implemented the recovery for $m \geq 16$. Below that, the matrices are too small for it to work cleanly. The low-rank terms that the intersections should cancel then overlap by chance and survive. The recovered kernel comes out larger than one-dimensional, so the attack cannot single out $\mathbf{u}$ and $\mathbf{v}$.

The attack in Section 6.3 is dominated by solving a linear system of size $\Theta(n^2)$ to find the bulk of $\widetilde{\mathbf{T}}$ in the first stage. This dominates all other operations and is generally $\mathcal{O}(n^{2\omega})$ for $\omega \in [2, 3]$, the linear algebra constant for matrix solvers. For the sizes we tested, $n \leq 32$ and hence $n^2 \leq 1024$, so a standard solver at $\omega = 3$ is preferable, rendering the cost $\mathcal{O}(n^6)$. The attack still runs in polynomial time and results are reported in Table 2. The attack always succeeds in recovering the message $b \in \{0, 1\}$ for $n > 4$. At $n = 4$, the observed nullity is greater than 1, and we cannot uniquely recover $\widetilde{\mathbf{T}}$ up to scale. Increasing the number of samples N_1 in the first stage did not resolve this issue.

Optimizations. Since the concrete cost of this attack is quite prohibitive in practice, we adopted some optimizations to make it faster to run. We used `FLINT` for the heavy lifting of native arithmetic on the BN254 base field and for the heavy matrix solver. We also used `RREF`, even though the kernel is guaranteed to be 1-dimensional by construction while recovering $\widetilde{\mathbf{T}}$, because our field is large and only `nmod_mat` exposes a direct `nullspace` API, while `fmpz_mod_mat` does not. Constructing the coefficients of the linear system directly in `FLINT` and with minimal data movement in memory also had a significant effect on performance. That ensured that the matrix solver dominated the cost as theoretically expected and that not much effort was wasted in tensor copies and rearrangements. We

Table 2: NSS attack timings over the BN254 scalar field, single-threaded. The reported time covers the full attack on the public ciphertext, that is, finding $\widetilde{\mathbf{P}}$ and $\widetilde{\mathbf{M}}$, evaluating $\widetilde{\mathbf{Q}}_\infty$ and $\widetilde{\mathbf{Q}}_{\text{aff}}$, finding their kernels, and recovering entries of $\widetilde{\mathbf{T}}$ until an encrypted message $b \in \{0,1\}$ is guessed. Attack timings are reported as an average of a total of ten attempts, five per encrypted bit $b \in \{0,1\}$, and for optimal choices $N_1 = 2n-1$ and $N_2 = n+2$.

Scheme	Circuit	n	k	N_1	N_2	Time (s)
NSS [BIJ+20]	Subset sum bit-checks	8	17	15	10	0.05
		12	25	23	14	0.2
		16	33	31	18	1
		24	49	47	26	7.3
		32	65	63	34	28.4

maintained most of the scheme generation and matrix manipulations in `Sage` to utilize its optimized matrix API.

7 Conclusion

We presented two attacks against witness encryption from affine determinant programs. The first exploits the sparsity of the underlying constraint system and recovers the encrypted message from the public matrices alone, while the second linearizes the nearly-skew-symmetric variant to recover the encrypted bit. Together with the reduction of Section 3, they cover all ADP-based witness encryption variants described in [SRG+26; BIJ+20], and we verified both in practice.

There remain several open problems for future work. First, it is unclear whether the commutator attack extends to dense circuits. Our distinguisher relies on the error terms $\mathbf{E}_i$ having low rank, which is no longer the case once every variable meets many gates. A complete answer would generalize the analysis from the sparse families we consider here to arbitrary circuits. Finally, it remains open whether the rank-based approach can be repaired, that is, whether one can construct a determinant-program-based witness encryption scheme that resists the attacks of this paper while retaining the concrete efficiency that makes these constructions attractive in the first place.

Acknowledgments. We thank Tarik Kaced, who independently arrived at the idea of using commutators and provided the first practical verification on a particular small instance of the AADP scheme as part of a published cryptographic challenge.

AI Tools Disclosure. We disclose that AI tools were used to assist with proofreading, in particular the detection of typos and grammatical errors, with discussing technical aspects relevant to the results, and with implementing the

practical experiments. The authors take full responsibility for the contents of this paper.

References

[BIJ+20] J. Bartusek, Y. Ishai, A. Jain, F. Ma, A. Sahai, and M. Zhandry. Affine Determinant Programs: A Framework for Obfuscation and Witness Encryption. In T. Vidick, editor, *ITCS 2020*, volume 151, 82:1–82:39. LIPIcs, January 2020 (cited on pages 2–15, 19, 23, 24, 26–28, 32).

[BIO+20] O. Barta, Y. Ishai, R. Ostrovsky, and D. J. Wu. On Succinct Arguments and Witness Encryption from Groups. In D. Micciancio and T. Ristenpart, editors, *CRYPTO 2020, Part I*, volume 12170 of *LNCS*, pages 776–806. Springer, Cham, August 2020 (cited on page 2).

[CV21] G. Choi and S. Vaudenay. Towards Witness Encryption Without Multilinear Maps - Extractable Witness Encryption for Multi-subset Sum Instances with No Small Solution to the Homogeneous Problem. In J. H. Park and S.-H. Seo, editors, *ICISC 21*, volume 13218 of *LNCS*, pages 28–47. Springer, Cham, December 2021 (cited on page 14).

[GGS+13] S. Garg, C. Gentry, A. Sahai, and B. Waters. Witness encryption and its applications. In D. Boneh, T. Roughgarden, and J. Feigenbaum, editors, *45th ACM STOC*, pages 467–476. ACM Press, June 2013 (cited on pages 2, 6).

[GLW14] C. Gentry, A. B. Lewko, and B. Waters. Witness Encryption from Instance Independent Assumptions. In J. A. Garay and R. Gennaro, editors, *CRYPTO 2014, Part I*, volume 8616 of *LNCS*, pages 426–443. Springer, Berlin, Heidelberg, August 2014 (cited on page 2).

[JLS21] A. Jain, H. Lin, and A. Sahai. Indistinguishability obfuscation from well-founded assumptions. In S. Khuller and V. V. Williams, editors, *53rd ACM STOC*, pages 60–73. ACM Press, June 2021 (cited on page 2).

[JLS22] A. Jain, H. Lin, and A. Sahai. Indistinguishability Obfuscation from LPN over $\mathbb{F}_p$, DLIN, and PRGs in NC^0. In O. Dunkelman and S. Dziembowski, editors, *EUROCRYPT 2022, Part I*, volume 13275 of *LNCS*, pages 670–699. Springer, Cham, 2022 (cited on page 2).

[RVV24] S. Ragavan, N. Vafa, and V. Vaikuntanathan. Indistinguishability Obfuscation from Bilinear Maps and LPN Variants. In E. Boyle and M. Mahmoody, editors, *TCC 2024, Part IV*, volume 15367 of *LNCS*, pages 3–36. Springer, Cham, December 2024 (cited on page 2).

[SRG+26] L. Soukhanov, Y. Rebenko, M. E. Gebali, M. Komarov, H. Kilinc-Alper, M. Abdalla, and P. Towa. Implementable Witness Encryption from Arithmetic Affine Determinant Programs. Cryptology ePrint Archive, Paper 2026/175, 2026. Revision date 2026-05-10 archived at `https://eprint.iacr.org/archive/2026/175/20260510:155808` (cited on pages 2–5, 8–10, 12, 14, 23, 27, 28, 32).

[Str83] V. Strassen. Rank and optimal computation of generic tensors. *Linear Algebra and its Applications*, 52/53:645–685, 1983 (cited on pages 5, 19).

[Tsa15] B. Tsaban. Polynomial-Time Solutions of Computational Problems in Noncommutative-Algebraic Cryptography. *Journal of Cryptology*, 28(3):601–622, July 2015 (cited on page 5).

[Tsa22] R. Tsabary. Candidate Witness Encryption from Lattice Techniques. In Y. Dodis and T. Shrimpton, editors, *CRYPTO 2022, Part I*, volume 13507 of *LNCS*, pages 535–559. Springer, Cham, August 2022 (cited on page 2).

[VWW22] V. Vaikuntanathan, H. Wee, and D. Wichs. Witness Encryption and Null-IO from Evasive LWE. In S. Agrawal and D. Lin, editors, *ASIACRYPT 2022, Part I*, volume 13791 of *LNCS*, pages 195–221. Springer, Cham, December 2022 (cited on page 2).

[YCY22] L. Yao, Y. Chen, and Y. Yu. Cryptanalysis of Candidate Obfuscators for Affine Determinant Programs. In O. Dunkelman and S. Dziembowski, editors, *EUROCRYPT 2022, Part I*, volume 13275 of *LNCS*, pages 645–669. Springer, Cham, 2022 (cited on page 4).

A General Linearization Attack on WE from CS

We describe a general setting in which a black-box linearization attack applies to specific compilers that turn a complete constraint system verification for NP (like R1CS) into a WE for all of NP.

Model a constraint system (CS) verification as a polynomial $\mathfrak{C} : \mathbb{F}_p^{n+1} \to \mathbb{F}_p^m$. This models the evaluation of m polynomial constraints on a projective assignment of n variables. The constraint system is satisfied at an assignment $\mathbf{x}$ if and only if $\mathfrak{C}(\mathbf{x}) = \mathbf{0}$. The degree of the constraint system is defined to be $d_{\mathfrak{C}} \coloneqq \deg(\mathfrak{C})$. $\mathfrak{C}$ is a constraint system for NP if finding a root of $\mathfrak{C}$ is NP-complete.

A compiler to turn a constraint system $\mathfrak{C}$ for NP into a WKEM for all of NP may be modeled as the evaluation of a *fixed* hidden polynomial $\mathfrak{H} : \mathbb{F}_p^m \to \{0,1\}^\lambda$ on the output of $\mathfrak{C}$. In other words, the decapsulation algorithm on any valid assignment $\mathbf{w}$ is modeled as $k \leftarrow \mathfrak{H}(r; \mathfrak{C}(\mathbf{w}))$, where all randomness is combined in r as (for example) the seed to generate the coefficients of $\mathfrak{H}$. Let $d_{\mathfrak{H}} \coloneqq \deg(\mathfrak{H})$ be the degree of the hidden polynomial. The decapsulation algorithm is then idealized as some oracle $\mathfrak{O}_r : \mathbf{w} \mapsto (\mathfrak{H}(r;\cdot) \circ \mathfrak{C})(\mathbf{w})$ that only takes an assignment and returns the function composition directly. Notice that the correct encapsulated key is the constant term of $\mathfrak{H}$.

Just to give a simple example, consider the R1CS constraint system

$$\mathfrak{C}_{R1CS} : \mathbf{w} \mapsto (\mathbf{A}\mathbf{w}) \circ (\mathbf{B}\mathbf{w}) - \mathbf{C}\mathbf{w}$$

and define a WKEM using the hidden polynomial

$$\mathfrak{H}_{R1CS} : \mathbf{e} \mapsto k + \sum_{i=0}^{m-1} r_i e_i$$

for $r_i \leftarrow \mathbb{F}_p$. This scheme simply masks the key by a random affine combination of the quadratic "gates".

Notice in that very simple example that an adversary $\mathcal{A}$ may adopt the following strategy:

1. for $j \in \{0, \ldots, m\}$, $\mathcal{A}$
   (a) samples a random assignment $\mathbf{w}^{(j)} \leftarrow \mathbb{F}_p^{n+1}$.

(b) computes the R1CS error $\mathbf{e}^{(j)} = \mathfrak{C}_{R1CS}(\mathbf{w}^{(j)})$. Notice that the circuit is public.
(c) calls the oracle (where r is fixed during encapsulation) to get $k^{(j)} = \mathfrak{O}_r(\mathbf{w}^{(j)})$.

2. $\mathcal{A}$ leverages that $k^{(j)} = \mathfrak{H}(r; \mathbf{e}^{(j)})$ to recover the coefficients of $\mathfrak{H}$ and in particular the key k, which is just the constant term $\mathfrak{H}(r; \mathbf{0})$.

Notice that $d_{\mathfrak{H}_{R1CS}} = 1$ and that is why only $m + 1$ oracle calls were needed to recover the key and it was in particular independent of $d_{\mathfrak{C}_{R1CS}} = 2$. The same technique would have worked exactly the same if we had for example a quartic constraint system $d_{\mathfrak{C}} = 4$ except for potentially more work to compute the errors in each iteration. Increasing the degree of $\mathfrak{H}$ has a more desirable effect because it increases the adversarial work exponentially in $d_{\mathfrak{H}}$ while an honest party calls the oracle only once on a valid assignment. Increasing the degree of $\mathfrak{C}$ however increases the work of both honest and adversarial parties equally and the gap is controlled only by $d_{\mathfrak{H}}$. The next lemma formalizes these observations.

Lemma 1 (Black-Box Linearization Attack on WKEM from Polynomial CS). Let λ be a security parameter. Let n be the size of the witness space (number of variables) and m be the number of polynomial constraints (gates). Let $\mathfrak{C}$ be a CS for NP with degree $d_{\mathfrak{C}} = \mathcal{O}(1)$. Let $\mathfrak{H}$ be a hidden deterministic polynomial of degree $d_{\mathfrak{H}}$ and with coefficients generated by a uniformly random seed $r \leftarrow \{0,1\}^*$. Consider a WKEM whose decapsulation algorithm is modeled by the oracle $\mathfrak{O}_r$ defined earlier as the function composition $\mathfrak{H} \circ \mathfrak{C}$ and where r is fixed during encapsulation. There exists an adversary $\mathcal{A}$ that succeeds in recovering the encapsulated key using $\mathcal{O}(m^{d_{\mathfrak{H}}})$ oracle calls and $\mathcal{O}(\lambda^c m^{c\, d_{\mathfrak{H}}})$ field operations, for some constant $2 \leq c \leq 3$.

Proof. The proof follows exactly the same template as for the simple WKEM from R1CS described above. The only difference in the general case is that for a degree $d_{\mathfrak{H}}$ polynomial in m variables there are $\binom{m+d_{\mathfrak{H}}}{d_{\mathfrak{H}}} = \mathcal{O}(m^{d_{\mathfrak{H}}})$ monomials. Therefore, $\mathcal{A}$ needs to make that many queries to $\mathfrak{O}_r$ and build a linear system of size $\lambda m^{d_{\mathfrak{H}}}$ justifying the work needed to recover the coefficients.

In each iteration, the adversary has to compute the errors, which takes

$$\mathcal{O}\left(m\binom{n + d_{\mathfrak{C}}}{d_{\mathfrak{C}}}\right) = \mathcal{O}(m\, n^{d_{\mathfrak{C}}})$$

operations. Making the reasonable assumption that $n = \Theta(m)$, the adversary needs $\mathcal{O}(m^{1+d_{\mathfrak{H}}+d_{\mathfrak{C}}})$ operations for that and we treat each oracle call as $\mathcal{O}(1)$ as they are typically negligible in comparison.

Solving the linear system, on the other hand, would take $\lambda^\omega m^{\omega d_{\mathfrak{H}}}$ where $2 \leq \omega \leq 3$ is a linear algebra constant and for a fixed CS degree $d_{\mathfrak{C}} = \mathcal{O}(1)$ the claim follows for $c = \omega$. □

Remark 3. Notice that an honest party makes a single oracle call and does not have to solve a linear system. Therefore, the gap is reasonably approximated by the number of oracle calls by the adversary, which is $\mathcal{O}(\text{poly}(\lambda))$ for any fixed $d_{\mathfrak{H}}$ and since typically $m = \text{poly}(\lambda)$.

Consequently, this black-box attack breaks any WKEM made from any such compiler whenever $d_{\mathfrak{H}} = \mathcal{O}(\text{polylog}(\lambda))$. The black-box attack, however, does not necessarily succeed as stated for $d_{\mathfrak{H}} = \Omega(\text{poly}(\lambda))$.

In particular, for the formula-based All-Accept ADP-based WE [BIJ+20, Section 5.2] and its AADP variant in [SRG+26, Construction 1] we have $\mathfrak{H} = \det(\cdot)$ since a valid witness maps to a determinantal variety of codimension 1 by construction. Therefore, $d_{\mathfrak{H}} = \Theta(m)$, which renders the black-box attack computationally infeasible to run for $m = \Omega(\lambda)$.

Remark 4. Oftentimes, the hidden polynomial $\mathfrak{H}$ is computed in two steps (as a function composition) where first a hidden low-degree polynomial is applied to compute some secret and then a public high-degree polynomial is applied to the secret to compute the actual key. A prototypical example is a hidden affine transformation applied to R1CS or QAP, then the output is hashed to compute the encapsulated key.

The complexity of the attack depends only on the hidden low-degree part. The adversary recovers the secret using the black-box attack on the low-degree phase, then applies the high-degree phase to the recovered secret once.